

Chapter 2

THE GENERAL INSTRUMENT CP1600

The CP1600 and the TMS 9900 were the first two NMOS 16-bit microprocessors commercially available. Even a superficial inspection of the CP1600 shows it to be more powerful than the National Semiconductor PACE (or 8900), **yet the CP1600 is not widely used.** This is because General Instrument does not support the CP1600 to the extent that National Semiconductor originally supported PACE, or most manufacturers support their 8-bit microprocessors.

General Instrument's marketing philosophy has been to seek out very high-volume customers; General Instrument supports low-volume customers only to the extent that this support would not require substantial investment on the part of General Instrument.

From the viewpoint of the low-volume microprocessor user, General Instrument's marketing philosophy is unfortunate. The CP1600 is an ideal microprocessor for the more sophisticated video games that are appearing, and its rich instruction set and capable architecture make it an ideal choice for data processing terminals and home computer systems. However, due to its limited support, potential low-volume CP1600 customers are likely to choose another equally capable product.

Three CP1600 parts are available, differentiated only by the clock speeds for which they have been designed.

The CP1600 requires a 3.3 MHz, two-phase clock and generates a 600 nanosecond machine cycle time.

The CP1600 requires a 4 MHz, two-phase clock and generates a 500 nanosecond machine cycle time.

The CP1610 requires a 2 MHz, two-phase clock and generates a 1 microsecond cycle time.

In addition to the CP1600 microprocessors themselves, the CP1680 Input/Output Buffer (IOB) is described in this chapter. Additional support devices for the CP1600 may be found in An Introduction to Microcomputers: Volume 3 — Some Real Support Devices.

The sole source for the CP1600 is:

GENERAL INSTRUMENT
Microelectronics Division
600 West John Street
Hicksville, New York 11802

There is no second source for the CP1600. General Instrument has a policy of discouraging second sources for its product line.

The CP1600 is fabricated using NMOS ion implant LSI technology; the device is packaged as a 40-pin DIP.

Three power supplies are required: +12V, +5V and -3V.

THE CP1600 MICROCOMPUTER SYSTEM OVERVIEW

Logic of our general microcomputer system which has been implemented by the CP1600 CPU is illustrated in Figure 2-1.

Observe that the CP1600 requires external logic to create its various timing and clock signals.

Some bus interface logic is shown as absent because a number of devices must surround the CP1600; these include:

- 1) An address buffer, since data and addresses are multiplexed on a single 16-bit bus.
- 2) Buffer amplifiers to provide the power required by the type of memory and I/O devices that will normally be connected to a CP1600 CPU.
- 3) A one-of-eight decoder chip to create eight individual control signals out of three controls output by the CP1600.
- 4) A one-of-sixteen multiplex chip to funnel sixteen external status signals into the CP1600 if using external branches.

Were you to compare Figure 2-1 with an equivalent figure for a low-end microprocessor such as the SC/MP (which is described in Chapter 3 of the Osborne 4 & 8-Bit Microprocessor Handbook (Osborne/McGraw-Hill, 1980), the CP1600 might appear to offer fewer logic functions; but within the functions it does provide, the CP1600 provides considerably more logic and program execution capabilities. Where low-end microprocessors choose to condense, onto a single chip, simple implementations of different logic functions, high-end products such as the CP1600 choose to provide more devices — with greater capabilities on each device.

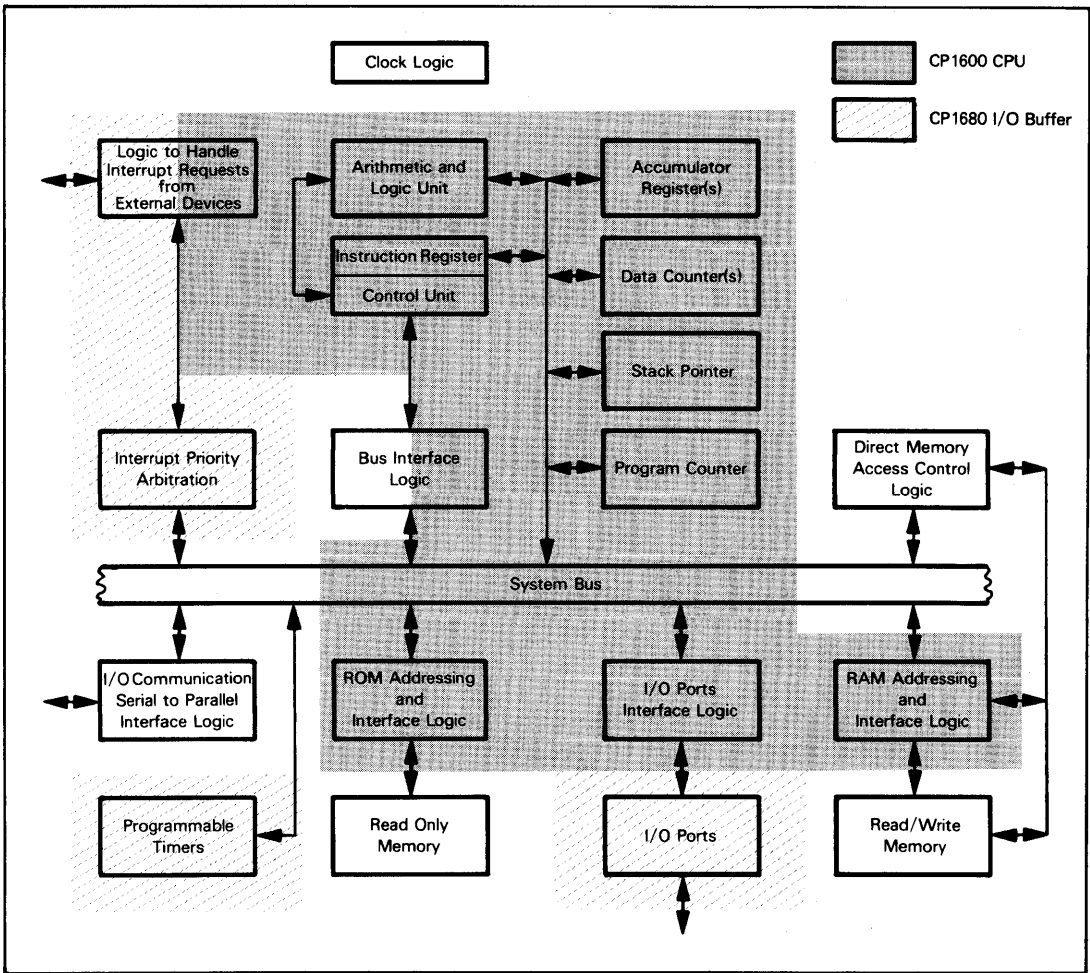
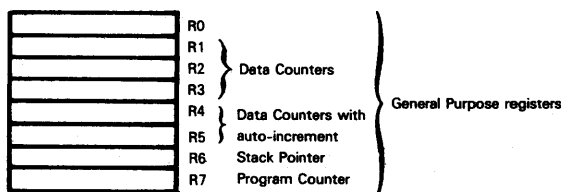


Figure 2 -1. Logic of the CP1600 CPU and CP1680 I/O Buffer

CP1600 PROGRAMMABLE REGISTERS

The CP1600 has eight 16-bit programmable registers, which may be illustrated as follows:



The way in which the registers illustrated above are used is unusual when compared to other microcomputers described in this book. All eight 16-bit registers can be addressed as though they were general purpose registers; however, only Register R0 has no other assigned function. We may therefore look upon Register R0 as the Primary Accumulator for this CPU.

Registers R1, R2, and R3 serve as general purpose registers, but may also be used as Data Counters.

In addition to serving as general purpose registers, R4 and R5 may be used as auto-incrementing Data Counters. Memory reference instructions that identify Register R4 or R5 as holding the implied memory address will cause the contents of Register R4 or R5 to be incremented — after the memory reference instructions have completed execution.

Registers R6 and R7, in addition to being accessible as general purpose registers, also serve as a Stack Pointer and a Program Counter, respectively.

Having the Stack Pointer accessible as a general purpose register makes it quite simple to maintain more than one Stack in external memory; also, you can easily address the Stack as data memory using the Stack Pointer as a Data Counter.

Having the Program Counter accessible as a general purpose register can be useful when executing various types of conditional branch logic.

While having the Stack Pointer and the Program Counter accessible as though they were general purpose registers may appear strange, this is a feature of the PDP-11 minicomputer — and is a very powerful programming tool.

CP1600 MEMORY ADDRESSING MODE

The CP1600 addresses memory and I/O devices within a single address space.

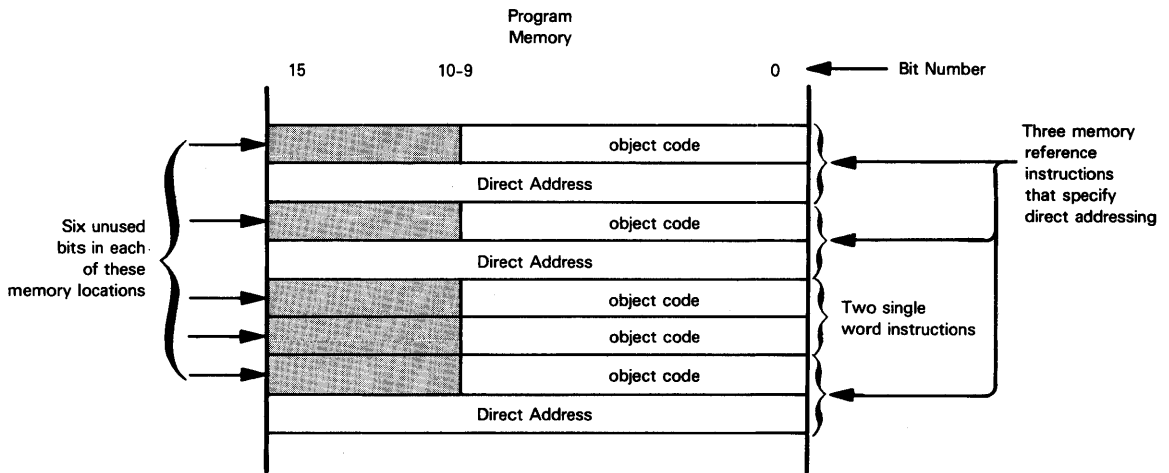
When referencing external memory, you can use direct addressing, implied addressing, or implied addressing with auto-increment.

Direct addressing instructions are all two or more words long, where the second or last word of the instruction object code provides a 16-bit direct address.

CP1600 DIRECT ADDRESSING

CP1600 direct addressing instructions are complicated by the fact that CP1600 program memory is frequently only 10 bits wide. That is to say, even though the CP1600 is a 16-bit microprocessor, its instruction object codes are only 10 bits wide. If program memory is only 10 bits wide, then direct addresses will only be 10 bits wide. A 10-bit direct address will access the first 1024 words of memory only.

Were you to implement a 16-bit wide program memory, then you could directly address up to 65,536 words of memory; however, six bits of the first object program word for every instruction in program memory would be wasted. This may be illustrated as follows:

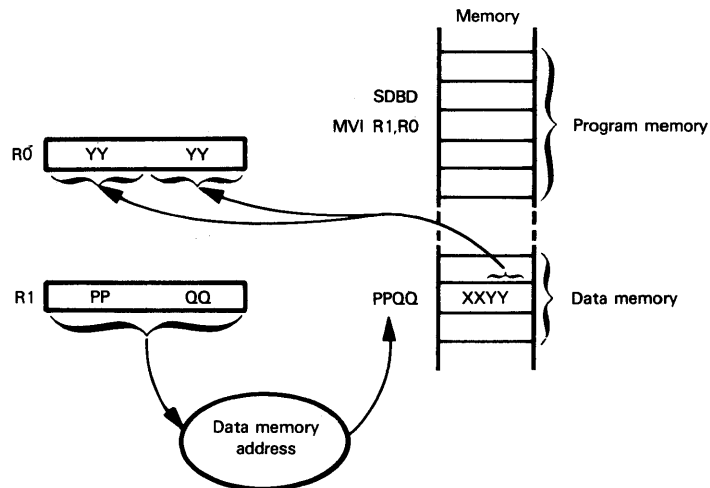


Instructions that reference memory using implied addressing identify general purpose Register R1, R2, or R3 as containing the implied address.

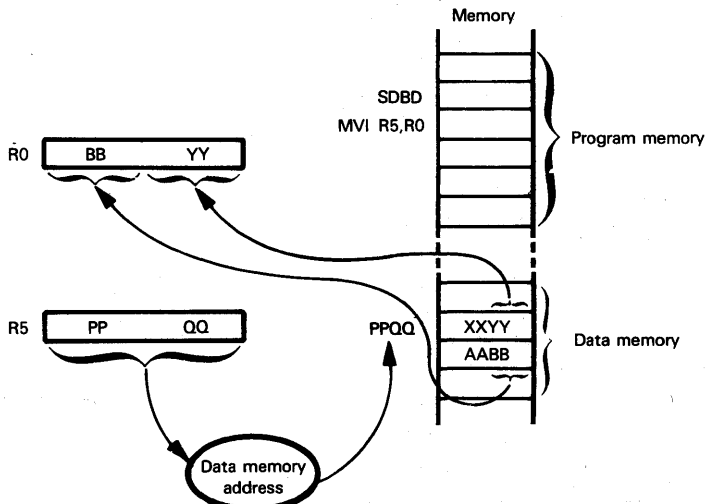
**CP1600
IMPLIED
ADDRESSING**

A memory reference instruction which identifies Register R4 or R5 as providing the external memory address will always cause Register R4 or R5 contents to be incremented following the memory access; thus you have implied memory addressing with auto-increment.

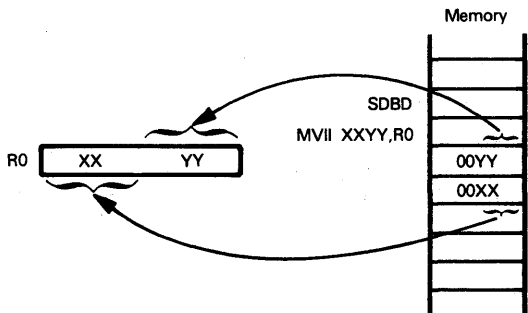
Memory reference instructions that specify implied memory addressing via Register 1, 2, 3, 4, or 5 can access 8-bit memory. An SDBD instruction executed directly before a valid memory reference instruction forces the memory reference instruction to access memory one byte at a time. If implied memory addressing via Register 1, 2, or 3 is specified, then the same byte of memory will be accessed twice. For an instruction that loads the contents of data memory into Register R0, this may be illustrated as follows:



If Register R4 or R5 provides the implied memory address for the instruction which follows an SDBD instruction, then the implied memory address is incremented twice, and two sequential low-order bytes of data are accessed. For an instruction which loads data into Register R0, this may be illustrated as follows:



The SDBD instruction may also precede an immediate instruction. Now the immediate data will be fetched from the low-order byte of the next two sequential program memory locations. This may be illustrated as follows:



Without the preceding SDBD instruction, an immediate instruction will access the next single program memory word to find the required immediate data. Ten or more bits of immediate data will be accessed, depending on the width of program memory words.

The CP1600 has no Stack reference instructions such as a Push or Pull; rather, a variety of memory reference instructions can identify Register R6 as providing the implied address.

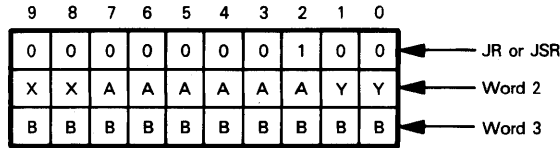
When Register R6 provides the implied address, it is treated as an upward migrating Stack Pointer. When a memory write operation specifies Register R6 as providing the implied memory address, Register R6 contents will be incremented following the memory write. A memory read instruction that specifies Register R6 as providing the implied memory address will cause the contents of Register R6 to be decremented before the read operation occurs.

**CP1600
STACK
ADDRESSING**

An unusual feature of the CP1600 is the fact that a variety of secondary memory reference instructions can also reference memory via the Stack Pointer. When these instructions are executed, Register R6 contents are decremented before the memory access occurs — as though a Pull operation from the Stack were being executed.

Logically, Register R6, the Stack Pointer, is being handled as though it were a Data Counter with post-increment and pre-decrement.

Jump instructions use direct memory addressing. Jump instructions are all three words long. The direct address is computed from the second and third memory words as follows:



AAAAAABBBBBBBBBB Jump address (binary)

yy are enable/disable bits for interrupts

xx identify the register where the return address will be stored for JSR

xx and yy are described in detail in Table 2-4.

You can enable or disable interrupts whenever you execute a Jump or Jump-to-Subroutine instruction.

The only difference between a Jump instruction and a Jump-to-Subroutine instruction is that the Jump-to-Subroutine instruction saves the Program Counter contents in Register 4, 5, or 6. The two high-order bits (xx) or the second Jump-to-Subroutine object code word specifies which of the three registers will be used to hold the return address.

Jump-to-Subroutine instructions, like the Jump instruction, allow direct memory addressing only.

CP1600 STATUS AND CONTROL FLAGS

The CP1600 CPU has four of the standard status flags; in addition, it has some unusual control signals.

These are the four standard status flags:

Sign (S). This status is set equal to the high-order bit of any arithmetic operation result.

Zero (Z). This status is set to 1 when any instruction's execution creates a zero result. The status is set to 0 for a nonzero result.

The Carry (C) and Overflow (O) statuses are standard carry and overflow, as described in Volume 1.

Four control signals (EBCA0 - EVCA3) are output during a Branch-on-External (BEXT) instruction. These four signals are output to reflect the low-order four bits of the BEXT instruction's object code. External logic receives these four signals and (depending on their state), may or may not return a high input via EBCI. If EBCI is returned high, then the BEXT instruction will perform a branch; if EBCI is returned low, then the BEXT instruction will cause the next sequential instruction to be executed. The four control signals EBCA0 - EBCA3 therefore provide the CP1600 with a means of testing 16 external conditions.

CP1600 CPU PINS AND SIGNALS

CP1600 CPU pins and signals are illustrated in Figure 2-2.

D0 - D15 is a multiplexed Address and Data Bus. Given a total of 40 pins in a package, CP1600 designers have been forced to share 16 pins between addresses and data. **Three control signals, BDIR, BC1, and BC2, identify the traffic on the Address/Data Bus. External logic (one MSI chip) must decode these three signals to create eight control signals, as summarized in Table 2-1.**

Remaining signals may be divided into four groups: timing, status/control, interrupt, and DMA.

Two timing clock signals are required: Φ1 and Φ2. These are complementary clock signals which may be illustrated as follows:



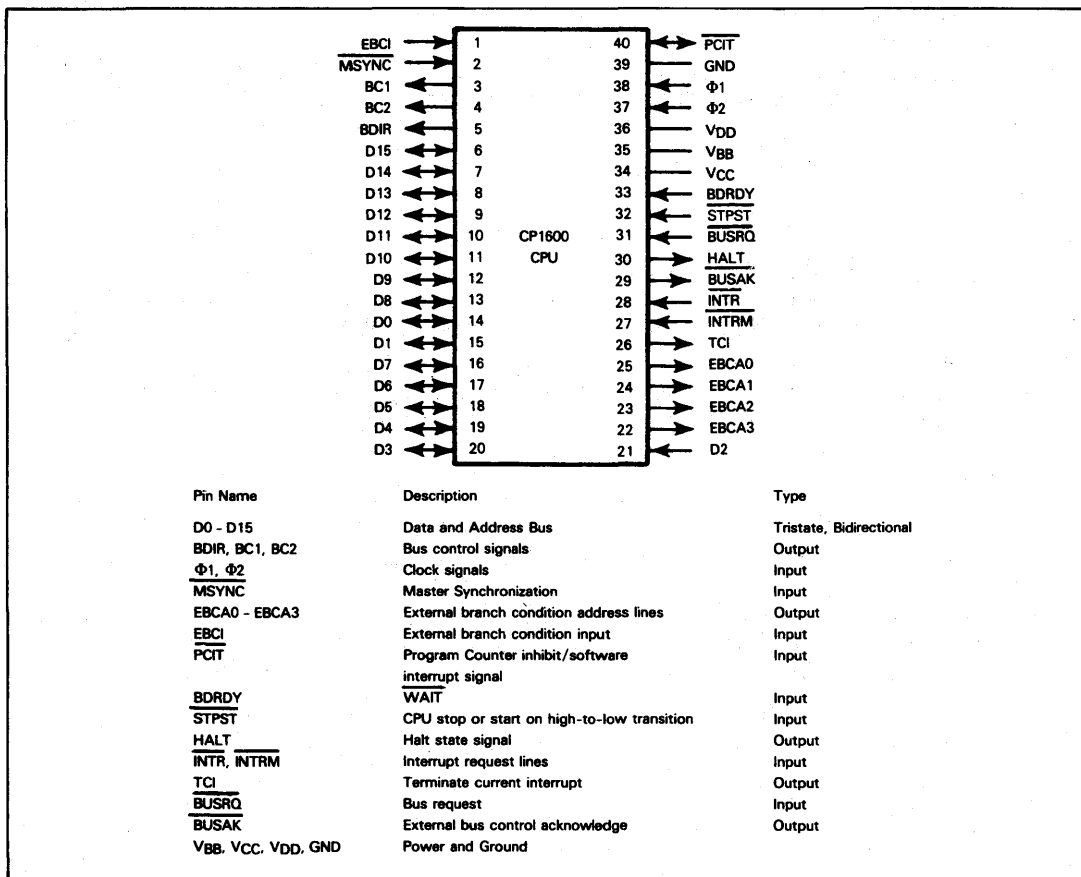


Figure 2-2. CP1600 CPU Signals and Pin Assignments

MSYNC is a somewhat unusual signal, as compared to other microcomputer clock signals in this book. Following powerup, MSYNC must be held low for at least 10 milliseconds. On the subsequent rising edge of MSYNC, logic internal to the CP1600 CPU will synchronize the $\Phi 1$ and $\Phi 2$ clock signals to start a new machine cycle. Most of the CPU devices we have described in this book use a reset signal, or have internal powerup logic which performs this clock synchronization.

Now consider the status and control signals.

First of all, **there are the four control outputs** which we have already described: **EBCA0 - EBCA3**. **There is one conditional Branch instruction (BEXT) which will only branch if a high signal is input via EBCI**. When the BEXT instruction is executed, the low-order four BEXT instruction object code bits are output via EBCA0 - EBCA3. External logic is supposed to decode these four signals by whatever means are appropriate — and thence determine whether EBCI should be input high or low. A high input, as we have just stated, will result in a branch; a low input will cause the next sequential instruction to be executed.

In reality, there is no connection within CP1600 CPU logic between the EBCI input and the four EBCA0 - EBCA3 outputs. So far as external logic is concerned, the execution of a BEXT instruction is identified by signal levels output and maintained on the EBCA0 - EBCA3 outputs, while the EBCI input determines whether a branch will or will not occur. How external logic chooses to determine whether EBCI will be set high or low is entirely up to external logic. The only vital function served by EBCA0 - EBCA3 is to identify the instant at which a BEXT instruction is executed.

Another unusual control signal provided by the CP1600 is PCIT; this is a bidirectional signal. When input low, this signal prevents the Program Counter from being incremented following an instruction fetch. This signal is also output as a low pulse following execution of a software interrupt instruction. Instruction timing separates the active input and

active output of this signal; providing external logic adheres to timing requirements, a conflict between input and output logic will never arise.

BDRDY is equivalent to the WAIT signal we have described for a number of other microcomputers. BDRDY is input low by any external logic which requires more time in order to respond to an I/O access. Recall that the CP1600 uses a single address space to reference memory or I/O devices. The BDRDY signal causes the CPU to enter a Wait state for as long as BDRDY is being input low; however, during the Wait state CPU logic is not refreshed. Thus a Wait state cannot last for more than 40 microseconds, or the contents of internal CPU locations will be lost.

STPST, a Halt/Reset input, is an edge-triggered signal. When external logic inputs a high-to-low transition via STPST, the CPU will complete execution of any interrupt instruction, then will enter a Halt state and output HALT high. If a non-interruptable instruction is being executed, then the Halt state will not begin until completion of next interruptable instruction's execution. The Halt state will last until external logic inputs another high-to-low STPST transition, at which time the Halt output will be returned low and normal programming execution will continue. Execution of the HLT instruction also causes the CP1600 to enter a Halt state, as described above.

Let us now look at interrupt signals.

The CP1600 has two interrupt request inputs — INTR and INTRM. INTR has higher priority than INTRM. INTR cannot be disabled. Typically, INTR will be used to trigger an interrupt upon power failure or other catastrophes.

The interrupt acknowledge signal is created by external logic which must decode the BC1, BC2, and BDIR signals, as shown in Table 2-1. Observe that there are, in fact, two interrupt acknowledge signals: the first (INTAK) acknowledges the interrupt itself, while the second (DAB) is used as a strobe for external logic to return an interrupt address vector. The interrupt sequence is described later in this chapter.

The CP1600 has two additional interrupt-related signals which are unusual when compared to other microcomputers described in this book.

TCI is output high when an End-of-Interrupt instruction is executed. This signal makes it easy for external logic to generate interrupt priorities which extend across the execution of an interrupt service routine.

Table 2-1. CP1600 Bus Control Signals

BC1	BC2	BDIR	SIGNAL	FUNCTION
0	0	0	NACT	The CPU is inactive and the Data/Address Bus is in a high impedance state.
0	0	1	BAR	A memory address must be input to the CPU via the Data/Address Bus.
0	1	0	IAB	Acknowledged external interrupt requesting logic must place the starting address for the interrupt service routine on the Address Bus.
0	1	1	DWS	Data write strobe for external memory.
1	0	0	ADAR	This signal identifies a time interval during which the Data/Address Bus is floated, while data input on the Data Bus is being interpreted as the effective memory address during a direct memory addressing operation.
1	0	1	DW	The CPU is writing data into external memory. DW will precede DWS by one machine cycle.
1	1	0	DTB	This is a read strobe which external memory or I/O logic can use in order to place data on the Data/Address Bus.
1	1	1	INTAK	This is an interrupt acknowledge signal. It is followed by IAD which is a strobe telling the external logic which is being acknowledged to identify itself by placing an address vector on the Data/Address Bus.

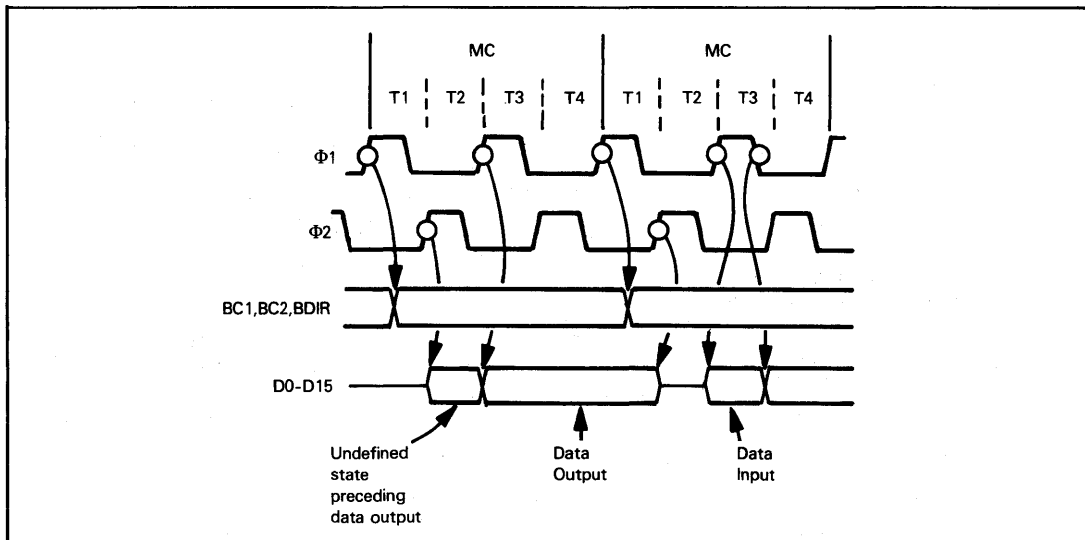


Figure 2-3. CP1600 Machine Cycles and Bus Timing

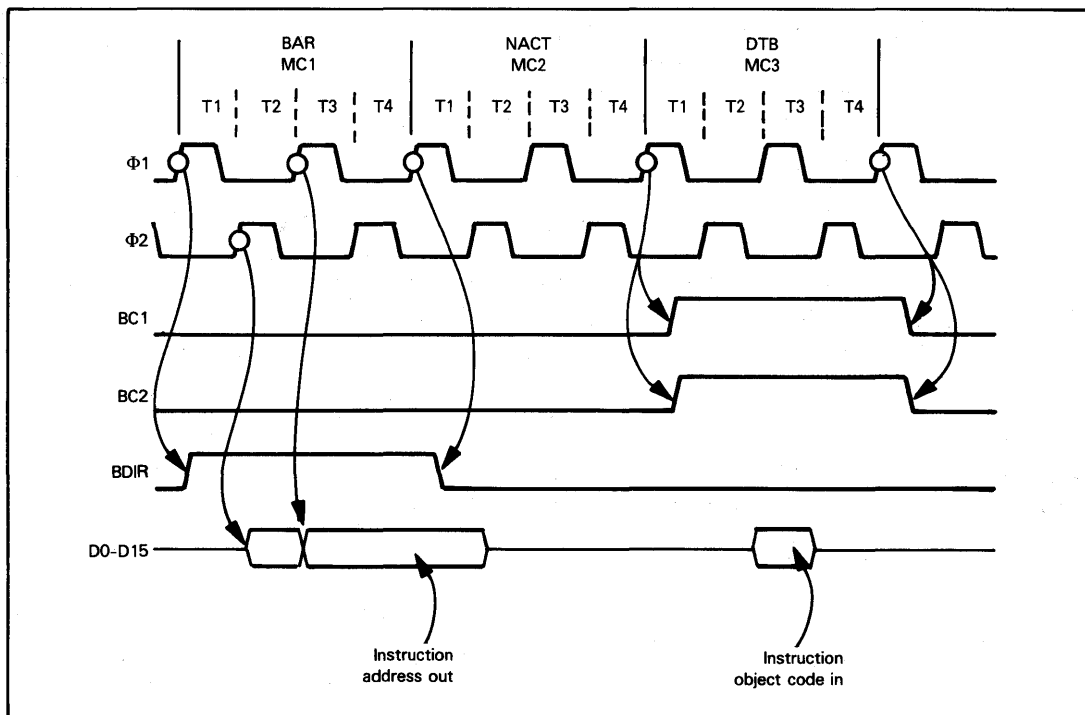


Figure 2-4. CP1600 Instruction Fetch Timing

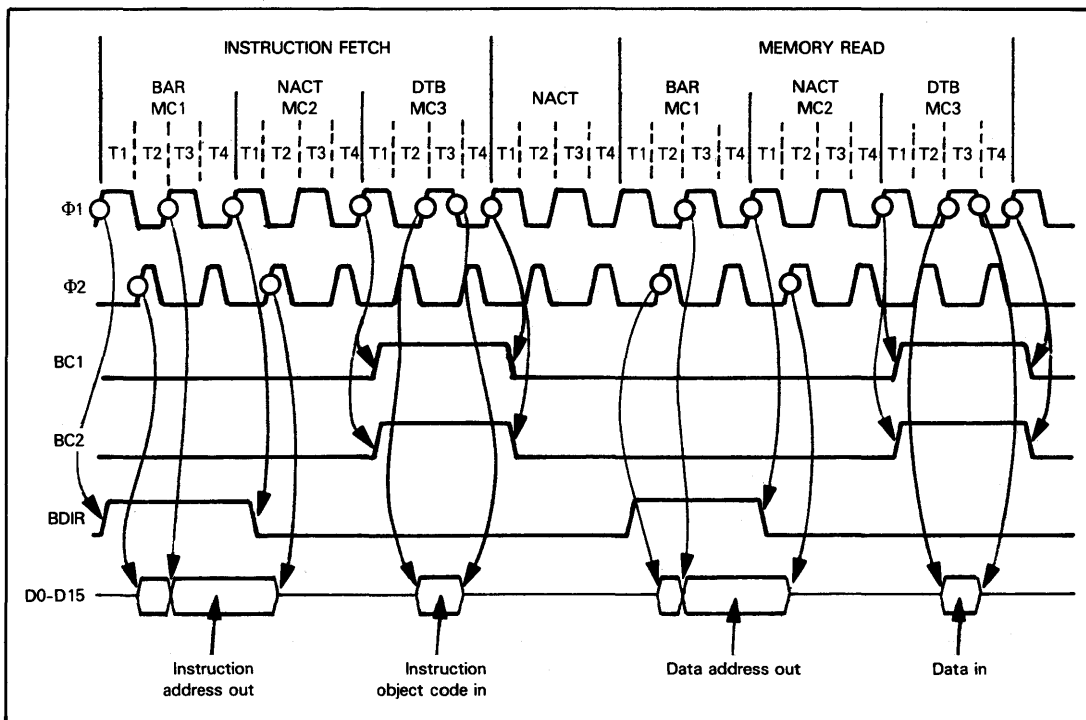


Figure 2-5. CP1600 Timing for Memory Read Instruction with Implied Memory Addressing

CP1600 INSTRUCTION TIMING AND EXECUTION

CP1600 instructions are executed as a sequence of machine cycles. Each machine cycle has four clock periods, as illustrated in Figure 2-3. Machine cycles are identified by their cycle number and by the levels of the BC1, BC2, and BDIR signals. Each of the eight level combinations is given a name, taken from Table 2-1. This name becomes the name of the machine cycle. Thus in Figure 2-4, and in subsequent instruction timing illustrations, **each machine cycle is identified by a signal name from Table 2-1.**

Figure 2-3 shows general case timing for data output or input on the Data/Address Bus. In between data input or output operations the bus is floated.

CP1600 MEMORY ACCESS TIMING

Figure 2-4 illustrates instruction fetch timing for a CP1600 instruction's execution. Three machine cycles are required. During the first machine cycle an address is output. Nothing happens during the second machine cycle; it is a "time spacing" machine cycle that routinely separates two CP1600 Bus access machine cycles. The object code for the accessed instruction is returned during the third machine cycle.

Figure 2-5 illustrates timing for the simplest memory read instruction's execution. In this case the data memory address is taken from one of the CPU registers. There is no difference between timing for the three machine cycles of an instruction fetch or a data memory read. As illustrated in Figure 2-5, a simple memory read instruction's execution consists of two three-machine cycle memory read operations, separated by a spacing no operation machine cycle.

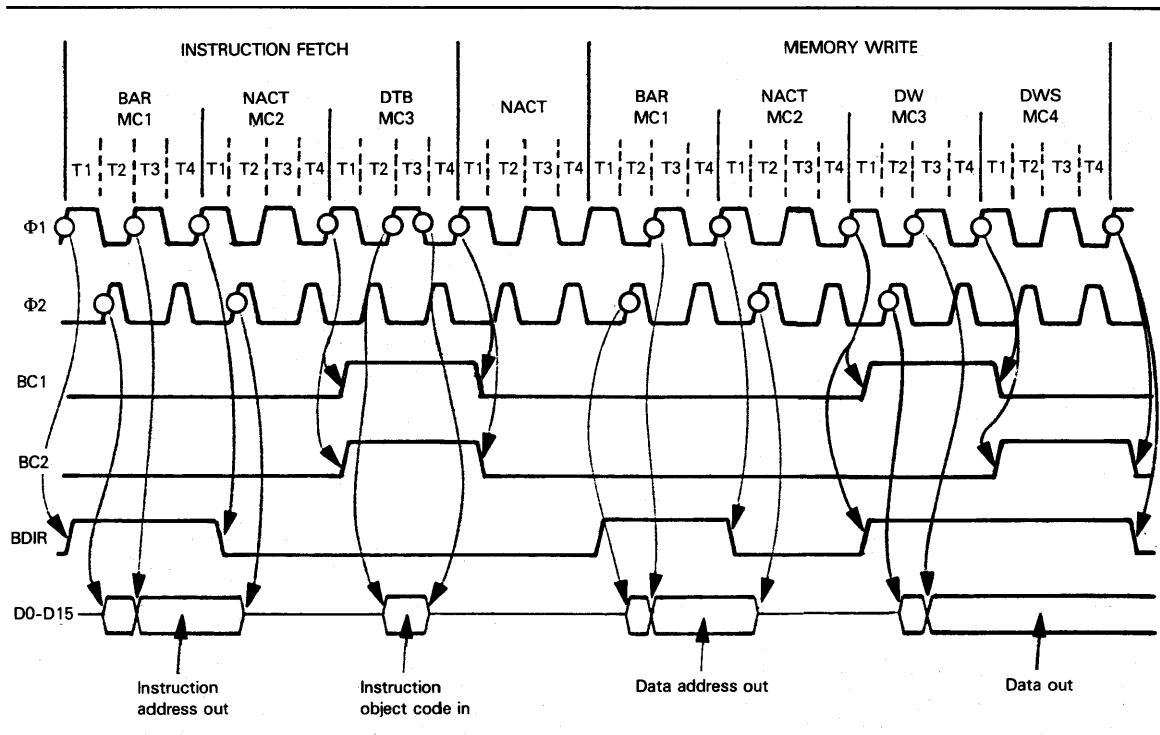
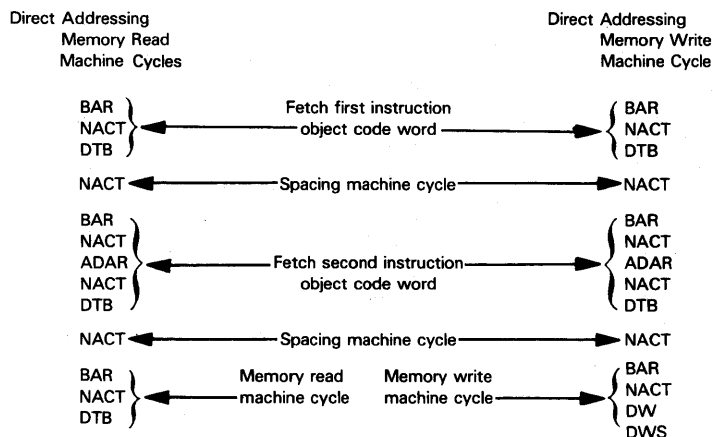


Figure 2-6. CP1600 Timing for Memory Write Instruction with Implied Memory Addressing

Figure 2-6 illustrates timing for a simple CP1600 memory write instruction execution. Data is output for two machine cycles, giving external logic ample time to respond to the data output. External logic uses the DWS machine cycle as a write strobe.

Any memory reference instruction that specifies direct memory addressing will require one three-clock-period machine cycle to fetch each word of the instruction object code; an NACT clock period will separate each machine cycle. After the first instruction fetch machine cycle, an ADAR-NACT clock period combination will be inserted in the second (and third, if present) instruction fetch machine cycle. During an ADAR clock period, BC1 is high, while BC2 and BDIR are low. No other control signals are active. Thus, **for a two-word memory read or memory write instruction that specifies direct addressing, the following clock periods and machine cycles will be required for instruction execution:**



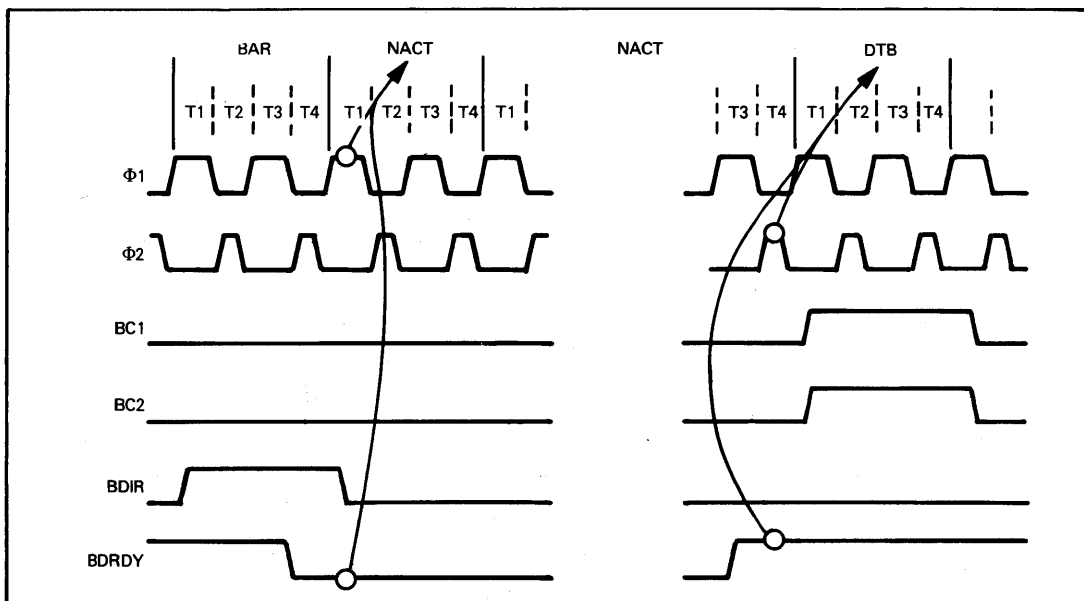


Figure 2-7. CP1600 Wait State Timing

THE CP1600 WAIT STATE

The CP1600 has a Wait state equivalent to those described for other microcomputers in this book. External logic that requires more time to respond to an access must input **BDRDY** low before the end of the BAR machine cycle, during which an address is output and the device is selected. **Timing is illustrated in Figure 2-7.**

If you examine Figures 2-4, 2-5 and 2-6, you will see that an address is output during a BAR machine cycle to initiate any external device access. The BAR machine cycle is always followed by an NACT machine cycle; in the middle of T1 during this NACT machine cycle, the CP1600 samples **BDRDY**. If **BDRDY** is low, then a sequence of NACT machine cycles occurs. In the middle of T4 for every NACT machine cycle, the CP1600 samples **BDRDY** again. Upon detecting **BDRDY** high, the CP1600 resumes instruction execution with a DTB machine cycle.

A Wait state must last for less than 40 microseconds, since the CP1600 is a dynamic device.

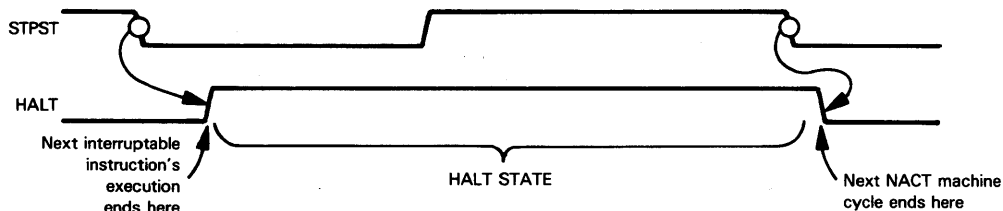
THE CP1600 HALT STATE

The CP1600 has a Halt state which may follow execution of the Halt instruction, or may be initiated by external logic.

When the Halt instruction is executed, then, following the instruction fetch machine cycle, the HALT signal is output high and a sequence of NACT machine cycles is executed.

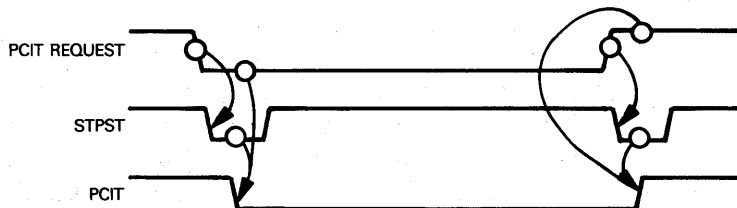
External logic initiates a Halt state by making the STPST input undergo a high-to-low transition. Following execution of the next interruptable instruction, a Halt state begins. The HALT signal is output high and a sequence of NACT machine cycles is executed.

A Halt state, whether it is initiated by execution of a Halt instruction or by a high-to-low transition of STPST, must be terminated by a high-to-low transition of STPST. This will cause the Halt state to end at the conclusion of the next NACT machine cycle. Timing for a Halt state which is initiated and terminated by STPST may be illustrated as follows:



The **PCIT** signal as an input inhibits CP1600 Program Counter increment logic. Thus, external logic can input **PCIT** low — in which case the same instruction will be continuously re-executed until **PCIT** goes high again. However, **PCIT** should only change levels while the CPU has been halted. Thus, **PCIT** and **STPST** should be used together as follows:

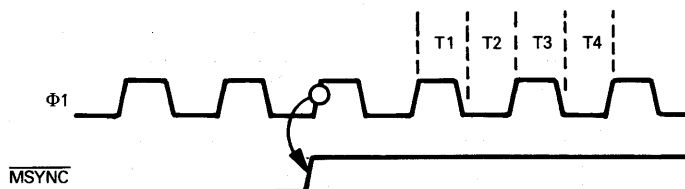
**CP1600
PCIT
SIGNAL**



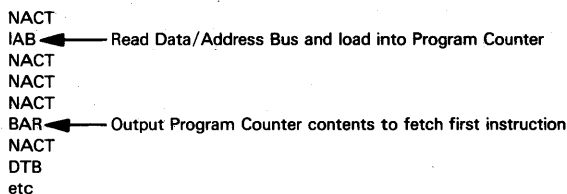
CP1600 INITIALIZATION SEQUENCE

The CP1600 is initialized by inputting the **MSYNC** signal low for a minimum of 10 milliseconds after power is first applied to the CPU.

MSYNC must make a low-to-high transition, marking the end of the initialization, on a rising edge of the $\Phi 1$ clock signal. On the next rising edge of $\Phi 1$, instruction execution will begin. This may be illustrated as follows:



When instruction execution begins, interrupts are disabled. The following sequence of machine cycles is executed:



During the IAB machine cycle, external logic must supply a 16-bit address at D0 - D15. Your external logic must provide this address, which in the simplest case may be 0000 by grounding the bus, or FFFF₁₆ by tying it to +5V following a startup.

The address which is input at IAB is output at BAR, initiating program execution.

CP1600 DMA LOGIC

CP1600 DMA logic is quite standard. When external logic wishes to transfer data under DMA control, it inputs **BUSRQ low. At the conclusion of the next interruptable instruction's execution, the CPU floats the Data/Address Bus and enters a Wait state, during which a sequence of NACT machine cycles is executed. **BUSAK** is output low at the beginning of the first NACT machine cycle.**

The NACT machine cycles that occur during a DMA operation refresh the CPU. NACT machine cycles that occur during a Wait state do not refresh the CPU. This means that any number of NACT machine cycles can occur during a DMA break, while a Wait state must be shorter than 40 microseconds.

The DMA break ends when external logic inputs **BUSRQ** high again. **BUSRQ** is sampled during T1 of every DMA NACT machine cycle. When **BUSRQ** is sampled high, two additional NACT machine cycles are executed, then **BUSAK** is output high and normal program execution resumes.

DMA timing is illustrated in Figure 2-8.

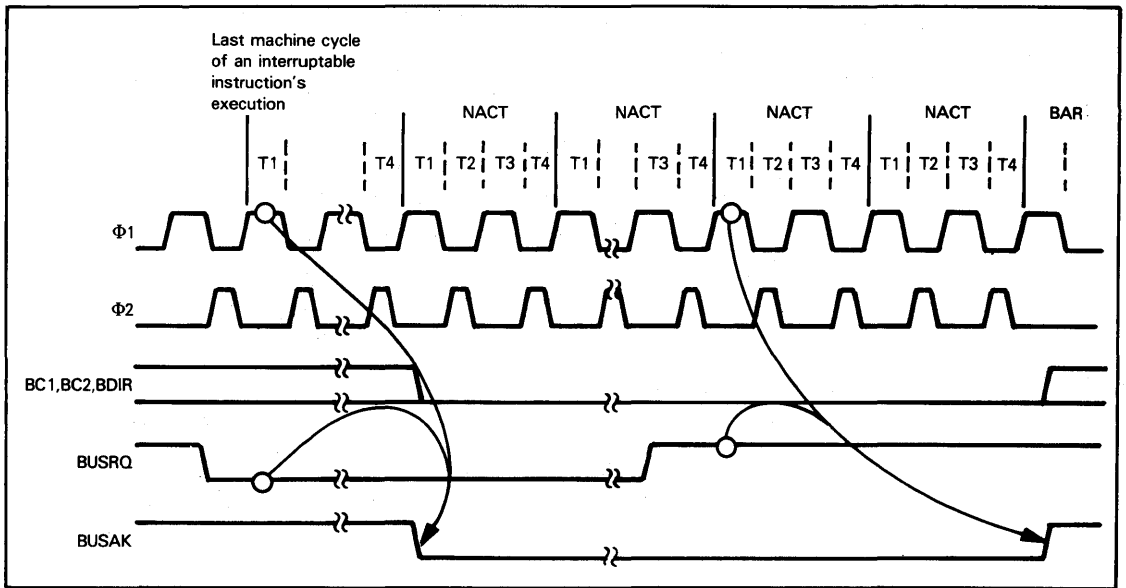


Figure 2-8. CP1600 DMA Timing

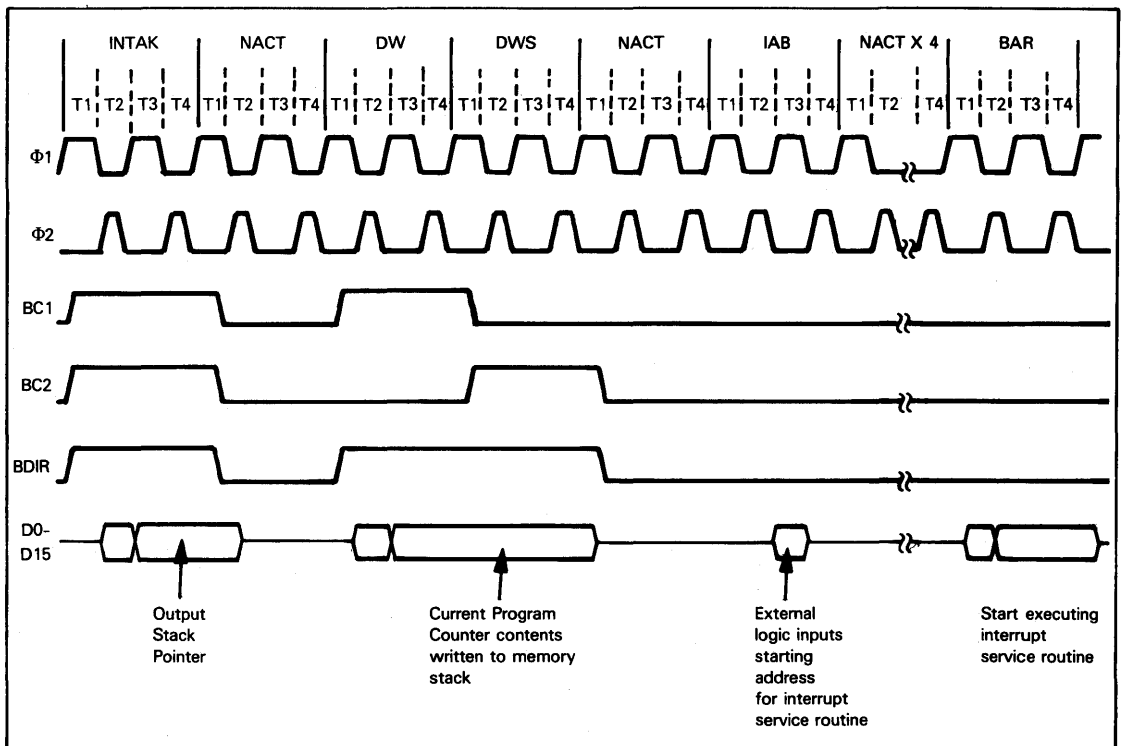


Figure 2-9. CP1600 Interrupt Service Routine Initialization

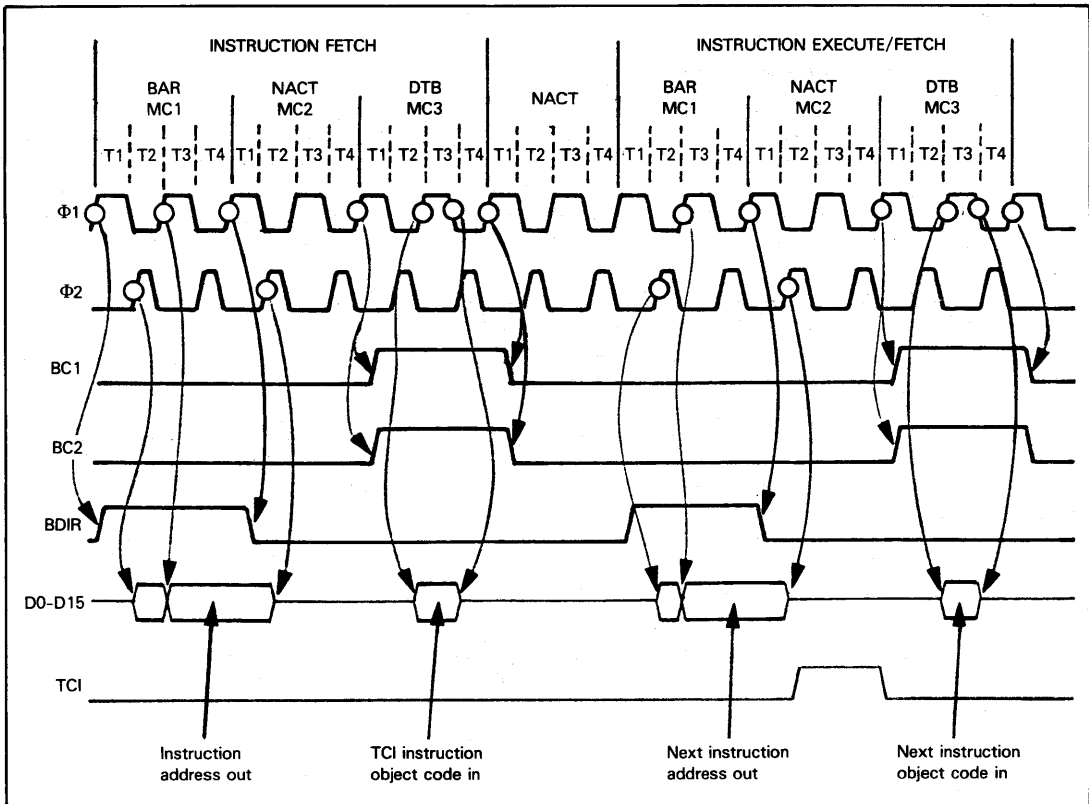


Figure 2-10. CP1600 Timing for TCi Instruction's Execution

THE CP1600 INTERRUPT LOGIC

The CP1600 uses a vectored interrupt processing system.

External logic requests an interrupt by inputting a low signal at either the $\overline{\text{INTR}}$ or $\overline{\text{INTRM}}$ pins.

Following the execution of the next interruptable instruction, the CP1600 acknowledges the interrupt by pushing Register R7 contents (the Program Counter) onto the Stack; then the CP1600 outputs 111, followed by 010 at BC1, BC2, and BDIR. External logic must respond by placing 16 bits of data on the Data/Address Bus. These 16 bits of data will be loaded into Register R7, the Program Counter, thus causing program execution to branch to an interrupt service routine dedicated to the interrupt. **Timing is illustrated in Figure 2-9.**

The $\overline{\text{PCIT}}$ signal is output low following execution of a software interrupt instruction (SIN). This is the only microcomputer described in this book which allows external logic to respond to a software interrupt in this fashion. Allowing external logic to respond to a software interrupt only makes sense when you anticipate your product being used in a minicomputer-like environment. Typically, the software interrupt will interface to logic of a front panel or console. When an SIN instruction is executed, a one-machine cycle low $\overline{\text{PCIT}}$ pulse is output.

You may, if you wish, end an interrupt service routine by executing a Terminate Current Interrupt (TCi) instruction, in which case the TCi signal will be output high.

Timing for TCi is given in Figure 2-10.

Following an interrupt acknowledge, the interrupt service routine must execute instructions in order to disable interrupts and save the contents of registers on the Stack. The exception is Register R7, the Program Counter, which is automatically pushed onto the Stack following an interrupt acknowledge.

External logic is entirely responsible for any type of interrupt priority arbitration which may occur, and for the generation of the interrupt vector address which must be input following an interrupt acknowledge.

It is quite easy to generate signals equivalent to other microcomputer system busses from the CP1600 System Bus. Therefore, you can use parts described in Volume 3 to handle CP1600 interrupt requirements.

THE CP1600 INSTRUCTION SET

The CP1600 instruction set is relatively straightforward. Addressing modes, which we have already described, are simple, and instructions are typical of those we have seen and described for other microcomputers. Unusual features relating to addressing modes available with **individual instructions are summarized in Table 2-2**, which describes the CP1600 instruction set.

If you have never programmed a PDP-11 minicomputer, then you should pay particular attention to programming techniques that result from the Stack Pointer and Program Counter being accessed as general purpose registers.

A wide variety of Register Operate instructions allow you to compute data and load the result directly into Register R7, the Program Counter. In effect, these become computed Jump instructions.

The ability to manipulate Register R6, the Stack Pointer, as though it were a general purpose register means that it is easy to maintain a number of different Stacks in external read/write memory.

The Jump-to-Subroutine instruction has a minicomputer flavor to it. Rather than saving the return address on the Stack, Register R7 contents are moved to General Purpose Register R4 or R5. A number of minicomputers will save a subroutine return address in a general purpose register in this fashion. The problem with this logic is that you must execute an additional instruction within the subroutine to save the return address on the Stack if you are going to use nesting subroutines. If you are passing subroutine parameters, however, this is an excellent arrangement, for the Jump-to-Subroutine instruction places the address of the parameter list directly in a Data Counter with auto-increment. We have described the concept of parameter passing in Volume 1, Chapter 7.

Note that the CP1600 instruction set lacks a logical OR.

In Tables 2-2 and 2-4, instruction length is given in terms of "words" rather than "bytes", as we have done in previous chapters. Since only the lower 10 bits of the CP1600 object code are presently used, system configurations need not have the full 16-bit word size. Hence a "word" may be 10 to 16 bits wide, depending on the implementation.

The following notation is used in Table 2-2:

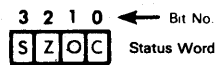
ADDR	One word of direct address
cond	Condition on which a branch may be taken. Table 1-3 lists all 14 branch conditions.
DATA	One word of immediate data.
DISP	One word displacement. See Table 2-4 for location of sign bit.
E	External branch condition.
EBCA0-3	The external branch condition address lines: EBCA0, EBCA1, EBCA2, and EBCA3.
EBCI	The external branch condition input line.
LABEL	A 16-bit direct address, target of a Jump instruction. See Table 2-4 for the bit format.
PCIT	The software interrupt output line.
RB	General Purpose Register R4, R5, or R6.
RD	One of the general purpose registers, used as a destination for operation results.
RM	One of the general purpose registers used as a Data Counter, R4 or R5, if specified, is auto-incremented after the memory access. R6 is incremented after a write, and decremented before a read.
RR	General Purpose Register R0, R1, R2, or R3.
RS	One of the general purpose registers, used as the source of an operand.

Statuses:

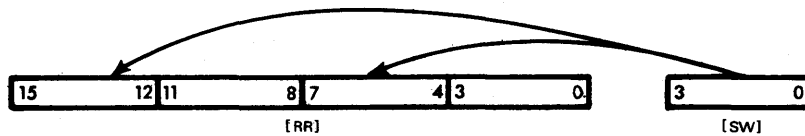
- S the Sign status
 - C the Carry status
 - Z the Zero status
 - O the Overflow status
- The following symbols are used in the STATUSES column:
- X the status flag is affected by the operation
 - a blank means the status flag is not affected
 - 0 the operation clears the status flag
 - 1 the operation sets the flag
 - 2 the Overflow flag is affected only on 2-bit shifts or rotates

SW

The Status Word, whose bits correspond to the condition of the status flags in the following way:



When the status word is copied into a register, it goes to the upper half of each byte:



When the status word is loaded from a register, it comes from the upper half of the lower byte:



- $x\langle y,z \rangle$ Bits y through z of the Register x . For example, $R7\langle 15,8 \rangle$ represents the upper byte of the Program Counter
- (.2) Indicates that the operand “.2” is optional
- A low pulse
- [] Contents of location enclosed within brackets. If a register designation is enclosed within the brackets, then the designated register's contents are specified. If a memory address is enclosed within the brackets, then the contents of the addressed memory location are specified.
- [[]] Implied memory addressing: the contents of the memory location designated by the contents of a register.
- $\wedge$ Logical AND
- ∇ Logical Exclusive-OR
- $\pm$ Addition or subtraction of a displacement, depending on the sign bit in the object code.
- $\leftarrow$ Data is transferred in the direction of the arrow.

Table 2-2. CP1600 Instruction Set Summary

TYPE	MNEMONIC	OPERAND(S)	WORDS	STATUSES	OPERATION PERFORMED
				S Z C O	
PRIMARY I/O AND MEMORY REFERENCE	MVI	ADDR,RD	2		[RD] ← [ADDR] Load register from memory, using direct addressing.
	MVI@	RM,RD	1		[RD] ← [[RM]] Load register from memory, using implied addressing.
	MVO	RS,ADDR	2		[ADDR] ← [RS] Store register to memory, using direct addressing.
	MVO@	RS,RM	1		[[RM]] ← [RS] Store register to memory, using implied addressing. If RS=R4, R5, R6 or R7, then RS=RM is not supported.
SECONDARY I/O AND MEMORY REFERENCE	ADD	ADDR,RD	2	X X X X	[RD] ← [RD] + [ADDR] Add memory contents to register, using direct addressing.
	ADD@	RM,RD	1	X X X X	[RD] ← [RD] + [[RM]] Add memory contents to register, using implied addressing.
	SUB	ADDR,RD	2	X X X X	[RD] ← [RD] - [ADDR] Subtract memory contents from register, using direct addressing.
	SUB@	RM,RD	1	X X X X	[RD] ← [RD] - [[RM]] Subtract memory contents from register, using implied addressing.
	CMP	ADDR,RS	2	X X X X	[RS] - [ADDR] Compare memory contents with registers, using direct addressing. Only the status flags are affected.
	CMP@	RM,RS	1	X X X X	[RS] - [[RM]] Compare memory contents with register's, using implied addressing. Only the status flags are affected.
	AND	ADDR,RD	2	X X	[RD] ← [RD] ∧ [ADDR] AND memory contents with those of register, using direct addressing.
	AND@	RM,RD	1	X X	[RD] ← [RD] ∧ [[RM]] AND memory contents with those of register, using implied addressing.
	XOR	ADDR,RD	2	X X	[RD] ← [RD] ⊕ [ADDR] Exclusive-OR memory contents with those of register, using direct addressing.
	XOR@	RM,RD	1	X X	[RD] ← [RD] ⊕ [[RM]] Exclusive-OR memory contents with those of register, using implied addressing.

Table 2-2. CP1600 Instruction Set Summary (Continued)

TYPE	MNEMONIC	OPERAND(S)	WORDS	STATUSES	OPERATION PERFORMED
				S Z C O	
IMMEDIATE	MVII	DATA,RD	2		[RD] ← DATA Load immediate to specified register.
	MVOI	RS,DATA	2		[(R7) + 1] ← [RS] Store contents of specified register in immediate field of MVOI instruction. This is only possible if program memory is read/write memory (rather than ROM).
IMMEDIATE OPERATE	ADDI	DATA,RD	2	X X X X	[RD] ← [RD] + DATA Add immediate to specified register.
	SUBI	DATA,RD	2	X X X X	[RD] ← [RD] - DATA Subtract immediate data from specified register.
	CMPI	DATA,RS	2	X X X X	[RD] - DATA Compare immediate data with contents of specified register. Only the status flags are affected.
	ANDI	DATA,RD	2	X X	[RD] ← [RD] ∧ DATA AND immediate data with contents of specified register.
	XORI	DATA,RD	2	X X	[RD] ← [RD] ⊕ DATA Exclusive-OR immediate data with contents of specified register.
JUMP	J	LABEL	3		[R7] ← LABEL Jump to given address.
	JR	RS	1	X X	[R7] ← [RS] Jump to address contained in specified register.
	JSR	RB,LABEL	3		[RB] ← [R7]; [R7] ← LABEL Jump to given address, saving Program Counter in R4, R5, or R6.
	B	DISP	2		[R7] ← [R7] + 2 ± DISP Branch relative to Program Counter contents.
BRANCH ON CONDITION	Bcond	DISP	2		If cond is true, [R7] ← [R7] + 2 ± DISP Branch relative on given condition; otherwise, execute next sequential instruction. EBCA0-3 ← E;
	BEXT	DISP,E	2		If EBCI=1, [R7] ← [R7] + 2 ± DISP Branch relative if external condition is true.

Table 2-2. CP1600 Instruction Set Summary (Continued)

TYPE	MNEMONIC	OPERAND(S)	WORDS	STATUSES	OPERATION PERFORMED
				S Z C O	
REGISTER-REGISTER MOVE AND OPERATE	MOVR	RS,RD	1	X X	[RD] ← [RS] Move contents of source register to destination register.
	ADDR	RS,RD	1	X X X X	[RD] ← [RS] + [RD] Add contents of specified registers.
	SUBR	RS,RD	1	X X X X	[RD] ← [RD] - [RS] Subtract contents of source register from those of destination register.
	CMPR	RS,RD	1	X X X X	[RD] - [RS] Compare registers' contents. Only the status flags are affected.
	ANDR	RS,RD	1	X X	[RD] ← [RD] ∧ [RS] AND contents of specified registers.
	XORR	RS,RD	1	X X	[RD] ← [RD] ⊕ [RS] Exclusive-OR contents of specified registers.
REGISTER OPERATE	CLRR	RD	1	0 1	[RD] ← [RD] V [RD] Clear specified register.
	TSTR	RS	1	X X	[RS] ← [RS] Test contents of specified register.
	INCR	RD	1	X X	[RD] ← [RD] + 1 Increment contents of specified register.
	DECR	RD	1	X X	[RD] ← [RD] - 1 Decrement contents of specified register.
	COMR	RD	1	X X	[RD] ← [RD] Complement contents of specified register (ones complement).
	NEGR	RD	1	X X X X	[RD] ← [RD] + 1 Negate contents of specified register (twos complement).
	ADCR	RD	1	X X X X	[RD] ← [RD] + [C] Add Carry bit to specified register contents.
	SLL	RR,(2)	1	X X	 [RR] Shift logical left one or two bits, clearing bit 0 (and bit 1 if shifting twice).

Table 2-2. CP1600 Instruction Set Summary (Continued)

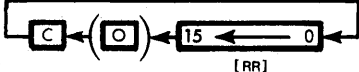
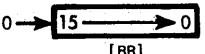
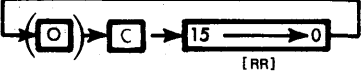
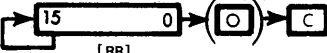
TYPE	MNEMONIC	OPERAND(S)	WORDS	STATUSES				OPERATION PERFORMED
				S	Z	C	O	
REGISTER OPERATE (CONTINUED)	RLC	RR(,2)	1	X	X	X	2	 Rotate left one bit through Carry, or rotate 2 bits left through Overflow and Carry.
	SLLC	RR(,2)	1	X	X	X	2	 Shift logical left one bit into Carry, clearing bit 0, or shift left two bits into Overflow and Carry, clearing bits 0 and 1.
	SLR	RR(,2)	1	X	X			 Shift logical right one or two bits, clearing bit 15 (and bit 14 if shifting twice).
	SAR	RR(,2)	1	X	X			 Shift arithmetic right one or two bits, copying high order bit.
	RRC	RR(,2)	1	X	X	X	2	 Rotate right one bit through Carry, or rotate two bits right through Overflow and Carry.
	SARC	RR(,2)	1	X	X	X	2	 Shift arithmetic right one bit into Carry, or two bits into Overflow and Carry.
	SWAP	RR(,2)	1	X	X			 Swap bytes of register once, or twice.

Table 2-2. CP1600 Instruction Set Summary (Continued)

TYPE	MNEMONIC	OPERAND(S)	WORDS	STATUSES	OPERATION PERFORMED
				S Z C O	
STACK	PSHR	RS	1		Separate mnemonics for MVO@ RS,R6.
	PULR	RD	1		Separate mnemonics for MVI@R6,RD.
INTERRUPT	SIN	(2)	1		PCIT — 
	EIS		1		Software interrupt.
	DIS		1		Enable interrupt system.
	TCI		1		Disable interrupt system.
	JE	LABEL	3		Terminate current interrupt.
	JD	LABEL	3		Jump to given address and enable interrupt system.
	JSRE	RB,LABEL	3		Jump to given address and disable interrupt system.
STATUS	GSWD	RD	1		[RD<15,12>]←[SW]; [RD<7,4>]←[SW] Place Status Word in upper half of each byte of the specified register. RD may be R0, R1, R2 or R3.
	RSWD	RS	1	X X X X	[SW]←[RS<7,4>] Load Status Word from bits 7 through 4 of the specified register.
	CLRC		1	0	[C]←0 Clear Carry.
	SETC		1	1	[C]←1 Set Carry.
	NOPP	(2)	2		No Operation.
	NOP		1		Halt after executing next instruction.
	HLT		1		Set double byte data mode for next instruction, which must be of one of the following types:
	SDBD		1		Primary or secondary I/O or memory reference Immediate or immediate operate If implied addressing through R1, R2, or R3 is used, the same byte will be accessed twice; addressing through R4, R5, or R7 will give bytes from the addressed location and that addressed after auto-increment. Direct addressing and Stack addressing are not allowed in double byte mode.

Table 2-3. CP1600 Branch Conditions and Corresponding Codes

MNEMONIC	BRANCH CONDITION	OBJECT CODE DESIGNATION
C	C = 1	0001
LGT	Carry (logical greater than)	
NC	C = 0	1001
LLT	No Carry (logical less than)	
OV	O = 1 Overflow	0010
NOV	O = 0 No overflow	1010
PL	S = 0 Plus	0011
MI	S = 1 Minus	1011
ZE	Z = 1 Zero (equal)	0100
EQ	Z = 0 Nonzero (not equal)	1100
NZE	$S \neq O = 1$ Less than	0101
LT	$S \neq O = 0$ Greater than or equal	1101
GE	$Z \vee (S \neq O) = 1$ Less than or equal	0110
LE	$Z \vee (S \neq O) = 0$ Greater than	1110
GT	C $\neq$ S = 1 Unequal sign and carry	0111
USC	C $\neq$ S = 0 Equal sign and carry	1111
ESC		

The following notation is used in Table 2-4:

Where ten digits are shown, they are the ten low-order bits of a 10 to 16-bit word. (Word size depends on the system implementation.) Where four digits are shown, they represent the hexadecimal notation for an entire word (10 to 16 bits).

bb Two bits indicating one of the first three general purpose registers:

00 = R0

01 = R1

10 = R2

cccc Four bits giving the branch condition, as shown in Table 2-3.

ddd Three bits indicating a destination register, RD:

000 = R0

001 = R1

010 = R2

011 = R3

100 = R4

101 = R5

110 = R6

111 = R7

eeee Four bits giving the external branch condition. E. Control signals EBCA0-EBCA3 reflect the state of these four bits.

IIII One word of immediate data (10 or 16 bits)

mmm	Three bits indicating a Data Counter Register RM: 000 = R0 001 = R1 010 = R2 011 = R3 100 = R4 101 = R5 110 = R6 111 = R7
m	One bit indicating the number of rotates or shifts: 0 one bit position 1 two bit positions
p	One bit of immediate address
P	One hexadecimal digit (4 bits) of immediate address
rr	Two bits indicating one of the first four general purpose registers: 00 = R0 01 = R1 10 = R2 11 = R3
sss	Three bits indicating a source register, RS: 000 = R0 001 = R1 010 = R2 011 = R3 100 = R4 101 = R5 110 = R6 111 = R7
z	Sign of the displacement: 0 add the displacement to PC contents 1 subtract the displacement from PC contents

In the "Machine Cycles" column, when two numbers are given with one slash between them (e.g., 7/9), execution time depends on whether or not a branch is taken. When two numbers are given, separated by two slashes (such as 8//11), execution time depends on which register contains the implied address.

THE BENCHMARK PROGRAM

For the CP1600 our benchmark program may be illustrated as follows:

	MVII	IOBUF,R4	LOAD THE I/O BUFFER STARTING ADDRESS INTO R4
	MVII	TABLE,R1	LOAD THE TABLE STARTING ADDRESS INTO R1
	MVI@	R1,R5	LOAD ADDRESS OF FIRST FREE TABLE WORD INTO R5
	MVII	CNT,R2	LOAD WORD COUNT INTO R2
LOOP	MVI@	R4,R0	LOAD NEXT DATA WORD FROM IOBUF
	MVO@	R0,R5	STORE IN NEXT TABLE WORD
	DECR	R2	DECREMENT WORD COUNT
	BNZE	LOOP	RETURN IF NOT END
	MVO@	R5,R1	RETURN ADDRESS OF NEXT FREE TABLE BYTE

This benchmark program makes very few assumptions. The input table IOBUF and the data table TABLE can have any length, and can reside anywhere in memory. The address of the first free word in TABLE is stored in the first word of the TABLE.

Table 2-4. CP1600 Instruction Set Object Codes

INSTRUCTION	OBJECT CODE	WORDS	MACHINE CYCLES
ADCR RD	0000101ddd	1	6
ADD ADDR,RD	1011000ddd PPPP	2	10
ADD@ RM,RD	1011mmddd	1	8//11
ADDI DATA,RD	1011111ddd IIII	2	8
ADDR RS,RD	0011ssddd	1	6
AND ADDR,RD	1110000ddd PPPP	2	10
AND@ RM,RD	1110mmddd	1	8//11
ANDI DATA,RD	1110111ddd IIII	2	8
ANDR RS,RD	0110ssddd	1	6
B DISP	1000z00000 PPPP	2	7/9
Bcond DISP	1000z0cccc PPPP	2	7/9
BEXT DISP,E	1000z1eeee PPPP	2	7/9
CLRC	0006	1	4
CLRR RD	0111ddddd	1	6
CMP ADDR,RS	1101000sss PPPP	2	10
CMP@ RM,RS	1101mmsss	1	8//11
CMPI DATA,RS	1101111sss IIII	2	8
CMPR RS,RD	0101ssddd	1	6
COMR RD	0000011ddd	1	6
DECR RD	0000010ddd	1	6
DIS	0003	1	4
EIS	0002	1	4
GSWD RR	00001100rr	1	6
HLT	0000	1	4
INCR RD	0000001ddd	1	6
J LABEL	0004 11pppppp00 PPPP	3	12
JD LABEL	0004 11pppppp10 PPPP	3	12
JE LABEL	0004 11pppppp01 PPPP	3	12
JR RS	0010sss111	1	7
JSR RB,LABEL	0004 bbpppppp00 PPPP	3	12
JSRD RB,LABEL	0004 bbpppppp10 PPPP	3	12

INSTRUCTION	OBJECT CODE	WORDS	MACHINE CYCLES
JSRE RB,LABEL	0004 bbpppppp01 PPPP	3	12
MOVR RS,RD	0010ssddd	1	6//7
MVI ADDR,RD	1010000ddd PPPP	2	10
MVI@ RM,RD	1010mmddd	1	8//11
MVII DATA,RD	1010111ddd IIII	2	8
MVO RS,ADDR	1001000sss PPPP	2	11
MVO@ RS,RM	1001mmsss	1	9
MVOI RS,DATA	1001111sss IIII	2	9
NEGR RD	0000100ddd	1	6
NOP (2)	000011010m	1	6
NOPP	1000z01000 PPPP	2	7
PSHR RS	1001110sss	1	9
PULR RD	1010110ddd	1	11
RLC RR,(2)	0001010mrr	1	6/8
RRC RR,(2)	0001110mrr	1	6/8
RSWD RS	0000111sss	1	6
SAR RR,(2)	0001101mrr	1	6/8
SARC RR,(2)	0001111mrr	1	6/8
SDBD	0001	1	4
SETC	0007	1	4
SIN (2)	000011011m	1	6
SLL RR,(2)	0001001mrr	1	6/8
SLLC RR,(2)	0001011mrr	1	6/8
SLR RR,(2)	0001100mrr	1	6/8
SUB ADDR,RD	1100000ddd PPPP	2	10
SUB@ RM,RD	1100mmddd	1	8//11
SUBT DATA,RD	1100111ddd IIII	2	8
SUBR RS,RD	0100ssddd	1	6
SWAP RR,(2)	0001000nrr	1	6/8
TCI	0005	1	4
TSTR RS	0010sssss	1	6//7
XOR ADDR,RD	1111000ddd PPPP	2	10
XOR@ RM,RD	1111mmddd	1	8//11
XORI DATA,RD	1111111ddd IIII	2	8
XORR RS,RD	0111ssddd	1	6

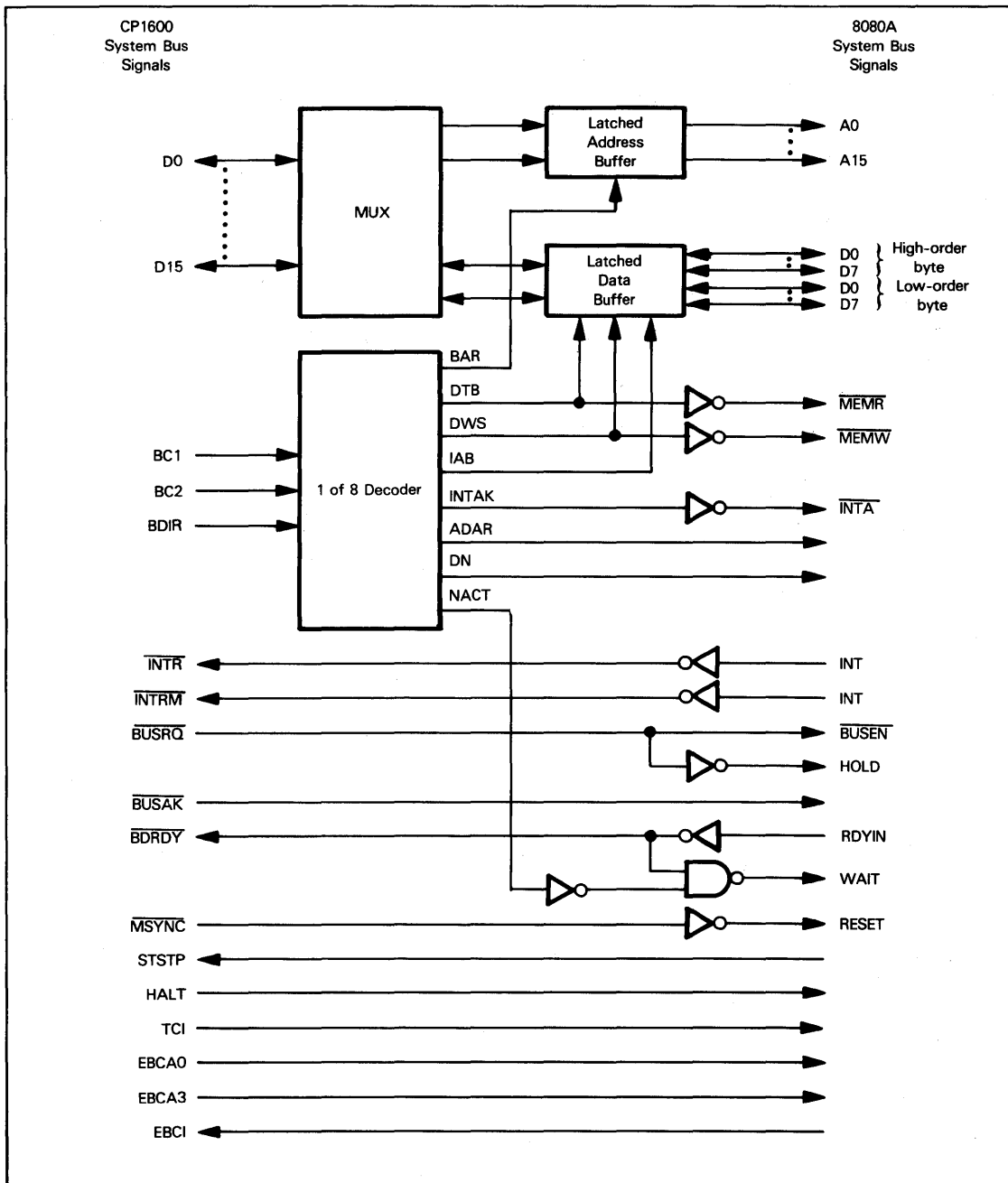


Figure 2-11. CP1600 to 8080A Bus Conversion

SUPPORT DEVICES THAT MAY BE USED WITH THE CP1600

A CP1600 microcomputer system with any significant capabilities will use support devices of some other microprocessor. Parallel I/O capability is available with the CP1680, (described next), but priority interrupt logic, DMA logic, and serial I/O logic, to mention just a few common options, may need additional support devices. Fortunately, **it is quite easy to generate an 8080A-compatible system bus from the CP1600 system bus. Logic is illustrated in Figure 2-11.**

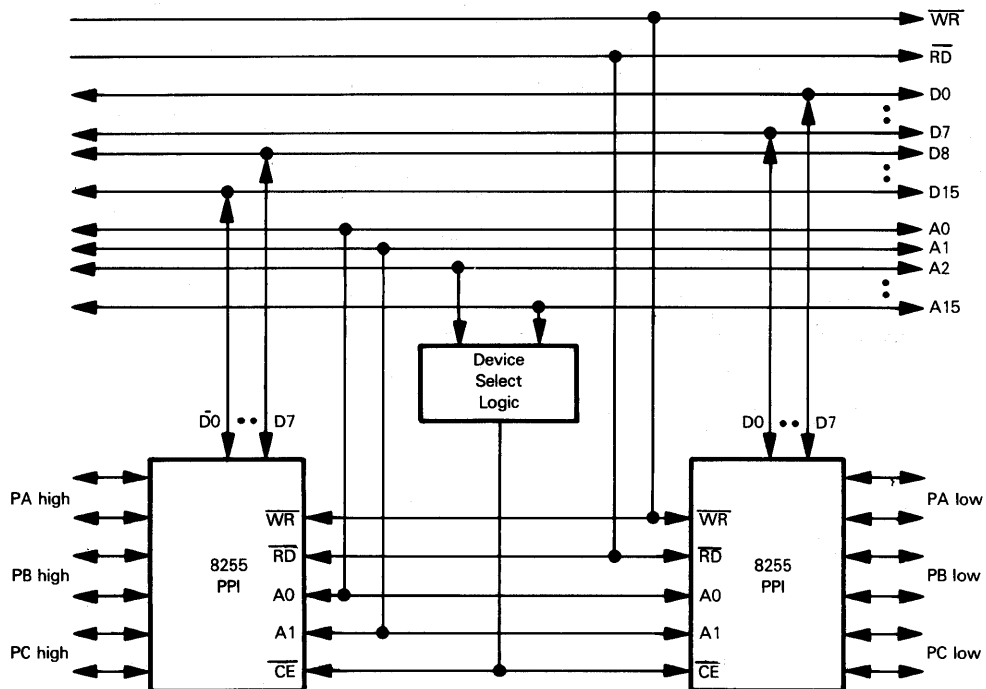
The CP1600A is the fastest version of the CP1600 CPU; it runs with a 500 nanosecond machine cycle. The CP1600 machine cycle is equivalent to an 8080A clock period. Since the standard 8080A clock period is also 500 nanoseconds, no speed conflicts will arise.

The bus-to-bus interface logic illustrated in Figure 2-11 is self-evident, with the exception of bus demultiplexing logic. The CP1600 Data/Address Bus is shown buffered by a demultiplexing buffer that is connected to two latched buffers. One of the latched buffers accepts the demultiplexer outputs only when a valid address is being output, as identified by BAR high. The second latched buffer may be a bidirectional latched buffer, or it may be two unidirectional latched buffers. Three latching strobes are required: DTB, IAB, and DWS.

DTB and IAB are data input strobes. DTB strobes data input that is to be interpreted as data, while IAB strobes data input that is to be interpreted as an address. So far as external logic is concerned, both of these signals are simple data input strobes. We could therefore generate a single data input strobe as the OR of DTB and IAB. When this data input strobe is high, information on the 8080A System Bus side of the latched data buffer must be input to the buffer; this data must simultaneously be transmitted to the multiplexer.

DWS is the data output strobe. When high, this signal must strobe data from the multiplexer to the latched data buffer; this latched data must immediately appear at the 8080A System Bus side of the latched data buffer.

Since the CP1600 uses a 16-bit Data Bus, you will probably have to generate two external device data busses; a high-order byte bus and a low-order byte bus. All external devices that transmit or receive parallel data must be present in duplicate. For example, were 8255 parallel interface devices to be present, the following connections would be required:



The CP1600 and MC6800 system busses are singularly incompatible. You should not attempt to use MC6800 support devices with the CP1600.

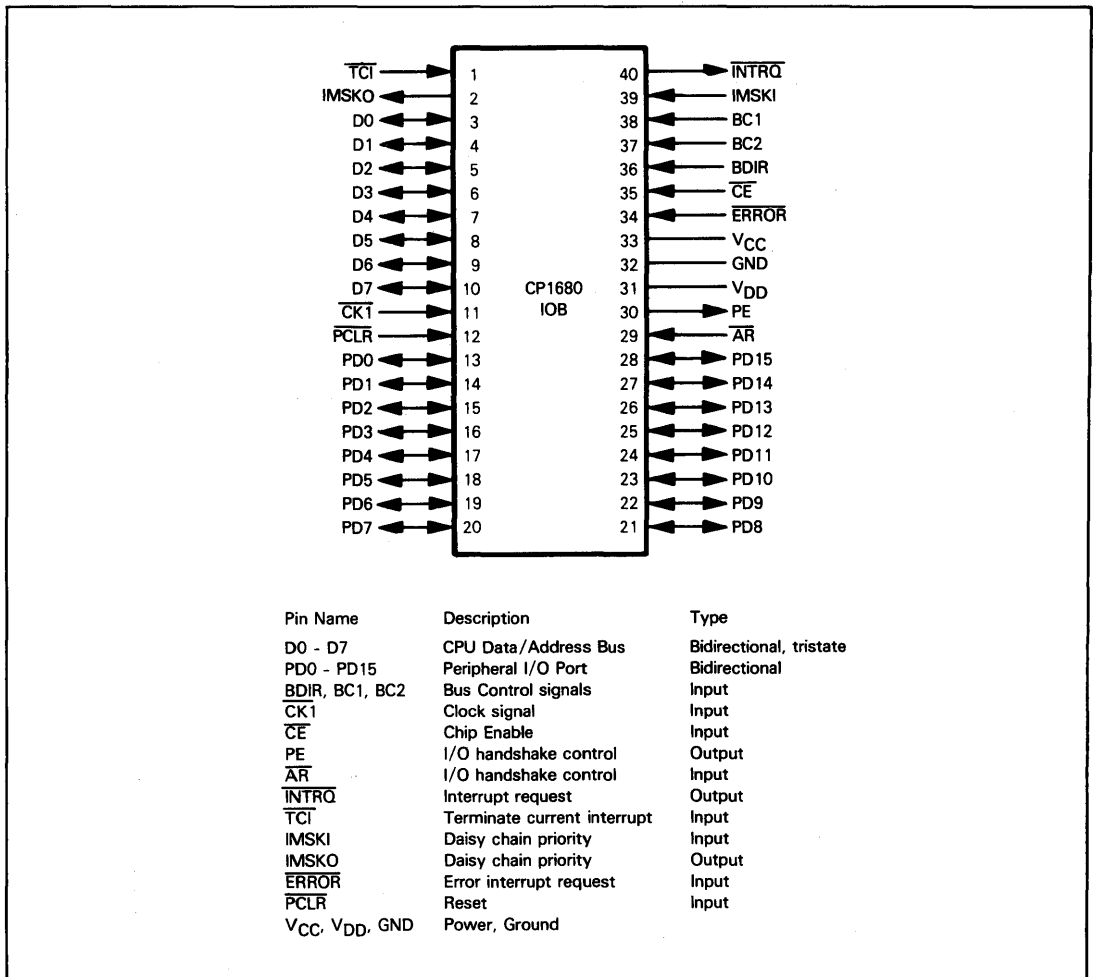


Figure 2-12. CP1680 IOB Signals and Pin Assignments

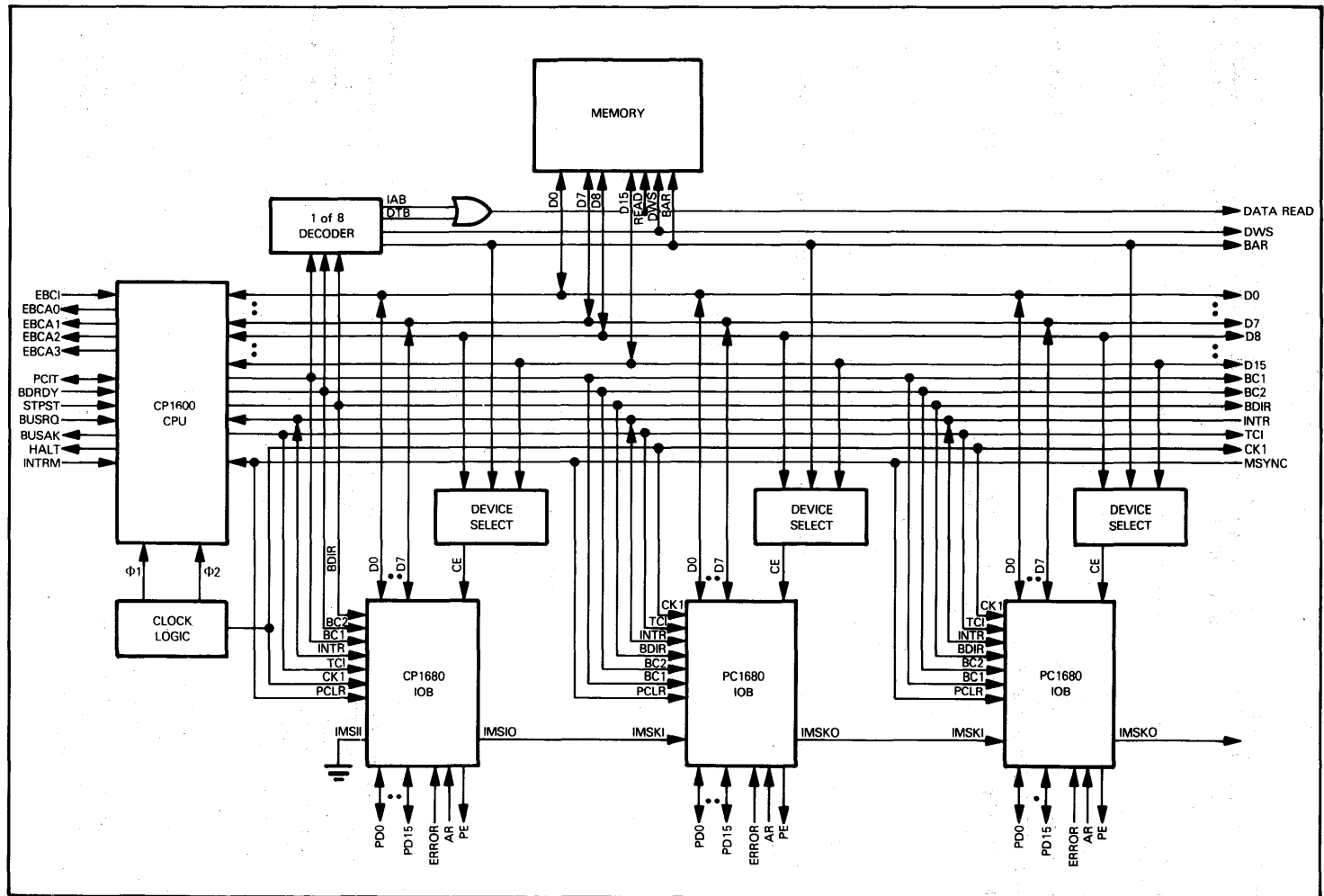


Figure 2-13. A CP1600-CP1680 Microcomputer Configuration

THE CP1680 INPUT/OUTPUT BUFFER (IOB)

The CP1680 IOB is a parallel I/O device designed specifically for the CP1600 CPU. This device provides a single 16-bit parallel I/O port, which may optionally be configured as two 8-bit I/O ports. Primitive handshaking control signals are available with the parallel I/O logic. Elementary interval timer and prioritized interrupt logic is also provided.

Figure 2-1 also illustrates that part of our general microcomputer system logic which has been implemented on the CP1680 IOB.

The CP1600 IOB is packaged as a 40-pin DIP. It requires two power supplies, +5V and +12V. All inputs are TTL compatible. The device is implemented using N-channel MOS technology.

Figure 2-13 illustrates a CP1600 microcomputer system with three CP1680 IOB devices in the configuration.

CP1680 IOB PINS AND SIGNALS

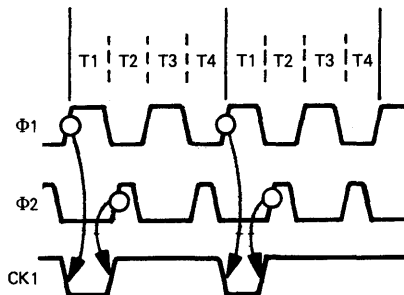
The CP1680 IOB pins and signals are illustrated in Figure 2-12. We will summarize these signals and the functions they serve before examining device operations in detail.

Let us begin by looking at the interface between the CP1680 IOB and the CP1600 CPU.

D0 - D7 provide an 8-bit parallel Data/Address Bus via which all communications between the CPU and IOB occur. This bus must connect to the low-order eight bits of the 16-bit CPU Data/Address Bus.

The three bus control signals, BC1, BC2, and BDIR, connect the CP1680 to the CP1600 as illustrated in Figure 2-13. The CP1680 IOB decodes these three bus control signals internally.

A clock input is required by the CP1680. This **clock input (CK1)** is used by internal logic to determine when BC1, BC2, and BDIR are valid. CK1 must have the following wave form:



CK1 must be derived from the CP1600 clock signals by external logic.

Let us now look at the interface between external logic and the CP1680 IOB.

PD0 - PD15 provide a 16-bit parallel I/O port which can optionally be configured as two 8-bit I/O ports. While PD0 - PD15 are in theory bidirectional, these pins are more accurately described as **pseudo-bidirectional**. This is because when a zero has been written to one of these pins, the output can sink 1.6 mA for an output voltage of +0.5V. External logic will have a hard time overcoming this sink in order to pull the pin high. In contrast, when a 1 is written to one of these pins, the output sources just 100 μ A at +5V. External logic will have little problem sinking 100 μ A in order to pull a pin low. Therefore, you should output a 1 to any pin that is subsequently to receive input data. External logic will then leave the pin high when inputting 1, while pulling the pin low to input 0.

**CP1600 I/O
PORT PIN
CHARACTERISTICS**

The handshaking control signals which link the CP1680 IOB with external logic are PE and $\overline{AR}$. PE is a control signal which is output by the CP1680, and $\overline{AR}$ is a control signal which is input to the CP1680.

Now consider CP1680 interrupt signals.

An interrupt request is transmitted to the CP1600 CPU via $\overline{INTRQ}$. The CPU acknowledges the interrupt via the **INTAK** combination of BDIR, BC1, and BC2. **TCl must be output low by the CPU at the end of the interrupt service routine.** This signal is required by CP1680 interrupt logic, which uses the low TCl pulse in its priority arbitration, as described later in this chapter.

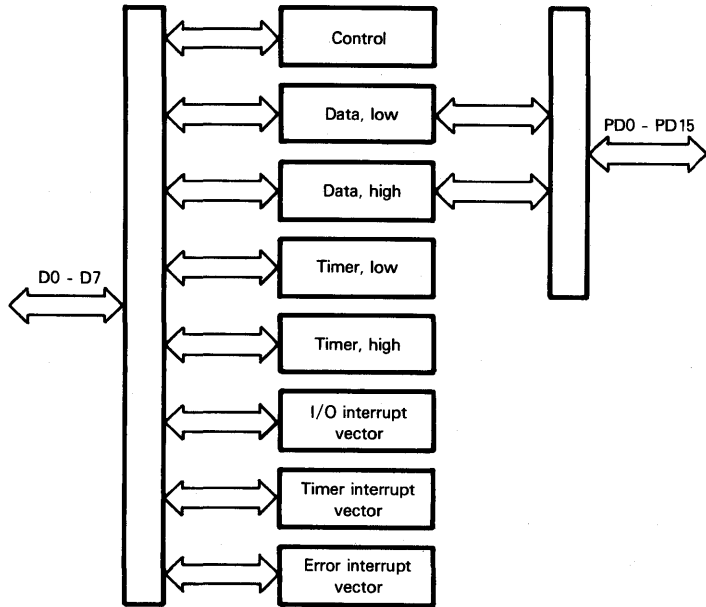
Interrupts may be generated by conditions internal to the CP1680, or by a low input at **ERROR**. The **ERROR** input is reserved for error conditions detected by external logic.

IMSKI and **IMSKO** are interrupt priority input and interrupt priority output signals, respectively. These signals are used to generate daisy chain interrupt priorities between CP1680 IOB devices, as illustrated in Figure 2-13. We will describe CP1680 interrupt priorities in more detail later in this chapter.

MCLR is the master reset control input for the CP1680. This signal must be input low for at least 10 milliseconds in order to reset the CP1680 IOB.

CP1680 ADDRESSABLE REGISTERS

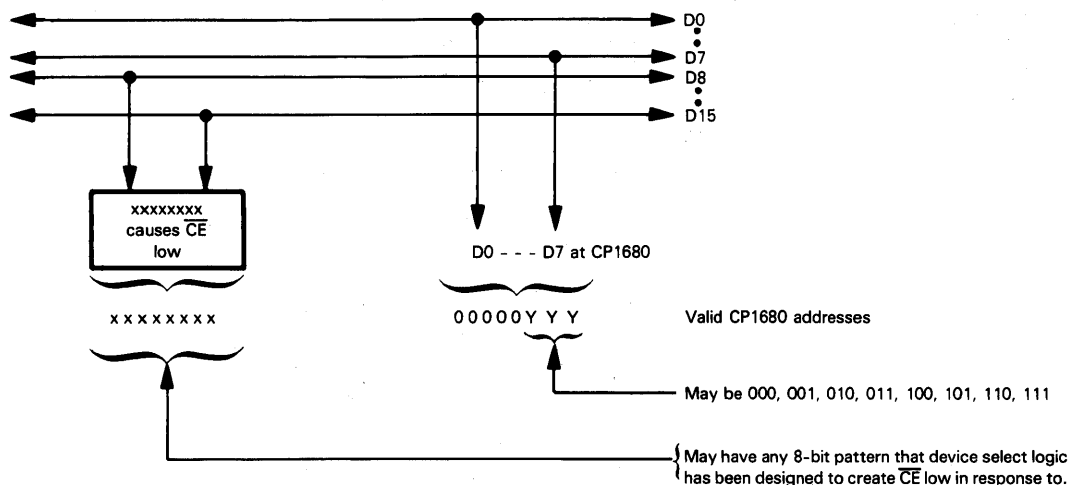
The CP1680 has eight addressable locations, which may be illustrated as follows:



These eight addressable locations are all 8-bit registers; they are addressed using the first eight addresses in a 256-address block, as follows:

Register	Address
Control	0
Data buffer, low-order byte	1
Data buffer, high-order byte	2
Timer, low-order byte	3
Timer, high-order byte	4
I/O interrupt vector	5
Timer interrupt vector	6
Error interrupt vector	7

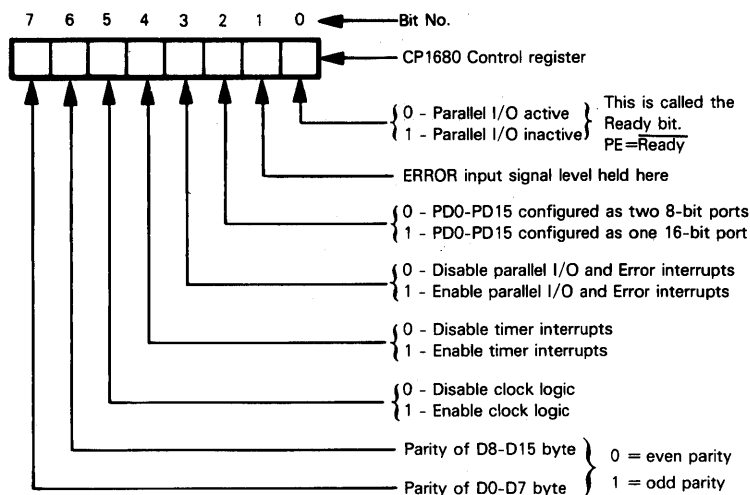
The actual 256 addresses will be identified by the eight high-order CP1600 Data/Address Bus lines, which will be used to create CP1680 device select logic. This device select logic creates $\overline{CE}$ (the chip enable signal); it may be illustrated as follows:



THE CP1680 CONTROL REGISTER

We will summarize the individual bits of the CP1680 control register before describing the operations they control.

Here are CP1680 Control register bit assignments:



Bit 0 is always the complement of the PE control output. This bit may be interrogated by the CPU. If parallel data transfer interrupts are disabled, this allows the CPU to poll on status when monitoring parallel data transfers. PE signal levels are illustrated in Figures 2-14 and 2-15.

Bit 1 reflects the level of the ERROR input. If parallel data transfer interrupt logic is disabled, then the Error interrupt logic is also disabled. Thus, the CPU must also examine the Error status bit when polling the CP1680.

Bit 2 determines whether PD0 - PD15 will act as a single 16-bit I/O port, or as two 8-bit I/O ports. This is only important when outputting data.

Control register bits 3 and 4 are used to enable and disable parallel data transfer and Error interrupt logic, and timer interrupt logic.

Control register bit 5 is used to enable and disable CP1680 interval timer logic. If this bit is 0, the interval timer will not decrement.

Bits 6 and 7 report the parity of the high-order byte and low-order byte for data that is input or output via PD0 - PD15. 0 indicates even parity while 1 indicates odd parity.

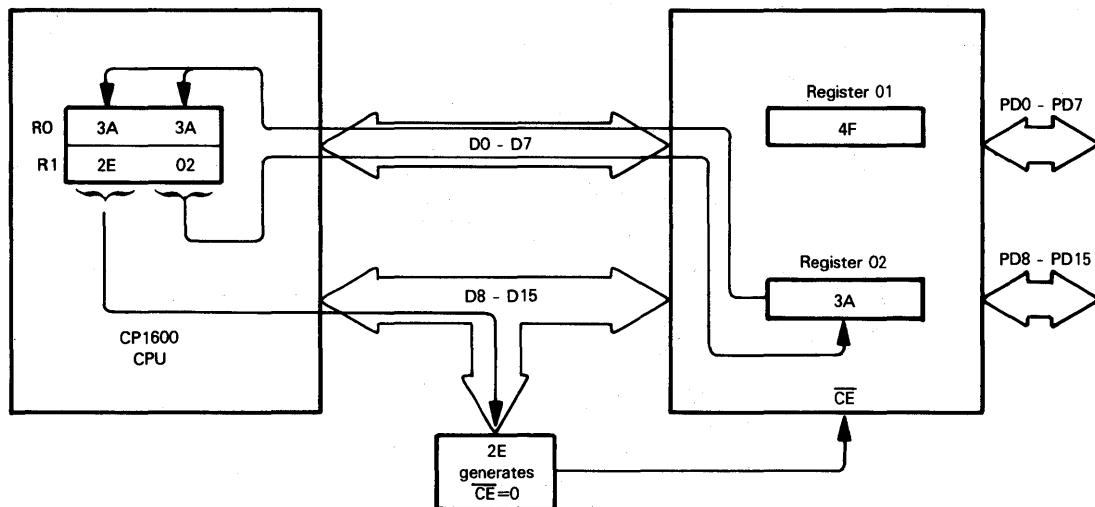
All Control register bits may be written into or read. You should be very careful when setting or resetting individual bits not to simultaneously modify other Control register bits. This means you should use a three-instruction sequence with an AND or OR mask to set or reset any Control register bit. For details see Volume 1, Basic Concepts.

CP1680 DATA TRANSFER OPERATIONS

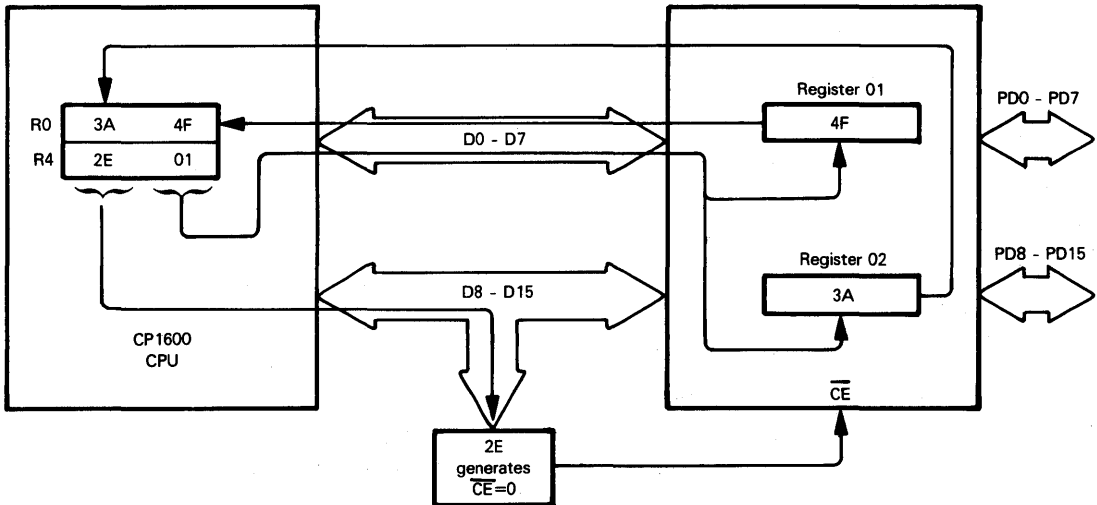
The CPU inputs and outputs data via the CP1680 IOB by executing MVI and MVO instructions, respectively.

The CPU must access the CP1680 in byte mode, since an 8-bit Data/Address Bus (D0 - D7) connects the CPU and the CP1680 IOB. Whether the I/O port PD0 - PD15 is configured as a single 16-bit port or as two 8-bit ports has no bearing on the fact that the CPU must access the CP1680 in byte mode.

The most efficient way of accessing the CP1680 is by using the SDBD instruction with implied memory addressing. Consider data input. If PD0 - PD15 is configured as two 8-bit I/O ports and you wish to access just one of these I/O ports, then you can use implied memory addressing via R1, R2, or R3. We may illustrate input from the high-order byte of I/O Port PD8 - PD15 as follows:



If PD0 - PD15 are configured as two 8-bit I/O ports or as a single 16-bit I/O port, and you want to read both I/O ports, then you should use the SDBD instruction with implied memory addressing via R4 or R5. This may be illustrated as follows:



Control register bit 2 configures PD0 - PD15 as a single 16-bit I/O port or as two 8-bit I/O ports.

Given the fact that MVI and MVO instructions (in byte mode) should be used to access the CP1680, when should these accesses occur?

The answer is that the PE and $\overline{AR}$ signals control event sequences.

Consider parallel data input, as illustrated in Figure 2-14.

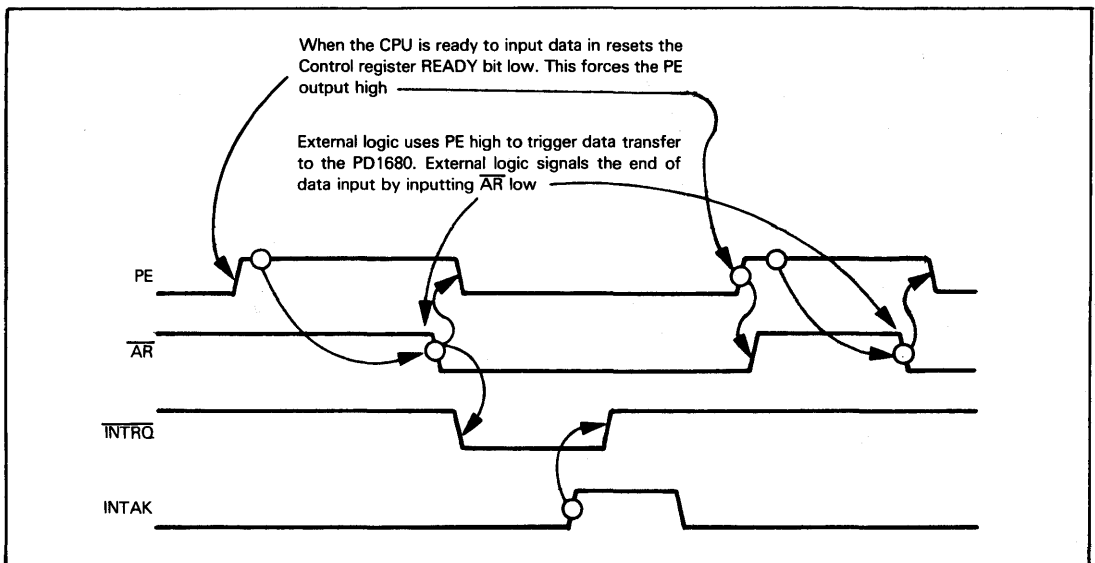


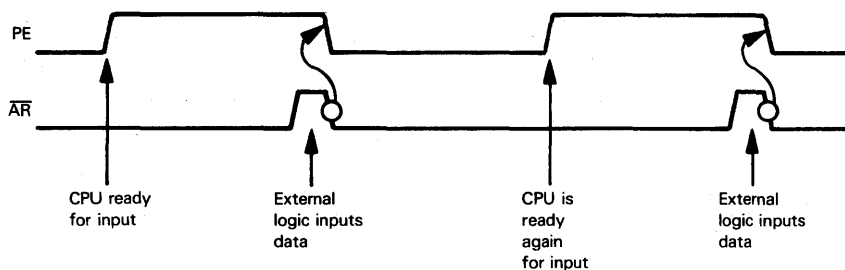
Figure 2-14. PD1680 Handshaking with Data Input

When the CPU is ready to receive data, it resets Control register bit 0 to 0; this forces the PE control signal high.

When external logic senses PE high, it must transmit data to the PD0 - PD15 I/O port. At this point it makes no difference whether pins have been configured as two 8-bit ports or as a single 16-bit port. External logic will pull to ground selected high pins, while leaving other high pins alone. When external logic has completed data input, it signals the fact by inputting $\overline{AR}$ low. It is the high-to-low transition of the $\overline{AR}$ control input which indicates the presence of new data for the CPU to read. When $\overline{AR}$ makes its high-to-low transition, PE also makes a high-to-low transition, and Control register bit 0 is set to 1. If interrupts have been enabled, then an interrupt is requested via $\overline{INTRQ}$. Figure 2-14 assumes that interrupts have been enabled; therefore $\overline{INTRQ}$ is shown making a high-to-low transition.

The CPU will acknowledge the interrupt request, as described earlier in this chapter, by outputting INTAK via BC1, BC2, and BDIR. Logic internal to the CP1680 uses INTAK to reset $\overline{INTRQ}$ high again.

There are many ways in which external logic can determine when to set $\overline{AR}$ high again. In Figure 2-14 we show external logic using PE to set $\overline{AR}$ high. Clearly, when PE makes a low-to-high transition, the CPU must have acknowledged $\overline{AR}$ low; therefore external logic can now set $\overline{AR}$ high. Now that $\overline{AR}$ is high again, external logic can input new data. An alternative scheme would be for external logic to constantly hold $\overline{AR}$ low, using the level of the PE output to determine when new data could be transmitted. When PE is high, external logic will transmit new data to the CP1680 once. As soon as it transmits new data, external logic will strobe the data with a short, high $\overline{AR}$ pulse, then wait for PE to go low and high again before inputting more data. This may be illustrated as follows:



Data output handshaking is illustrated in Figure 2-15.

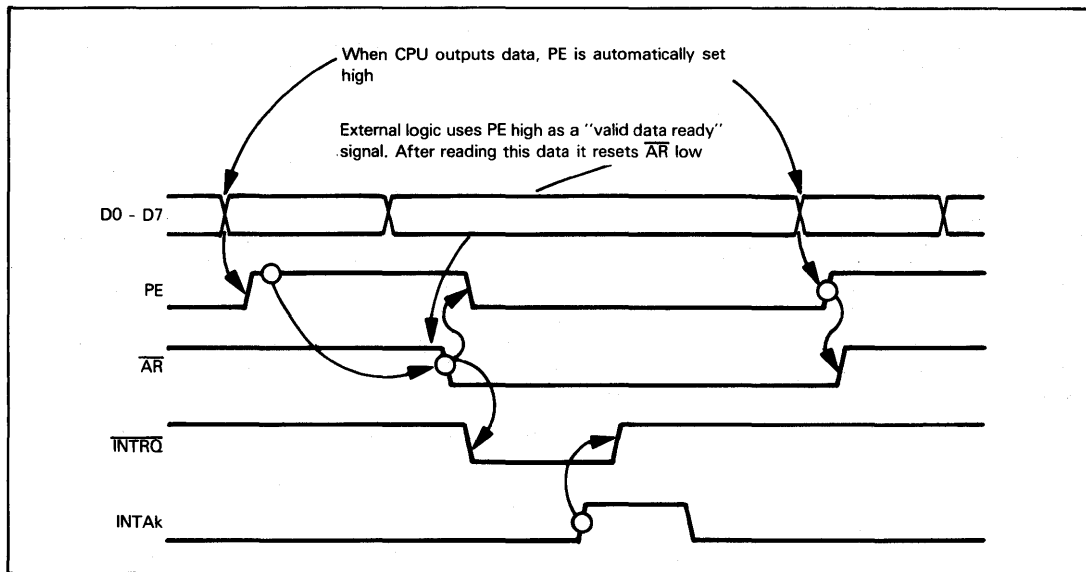


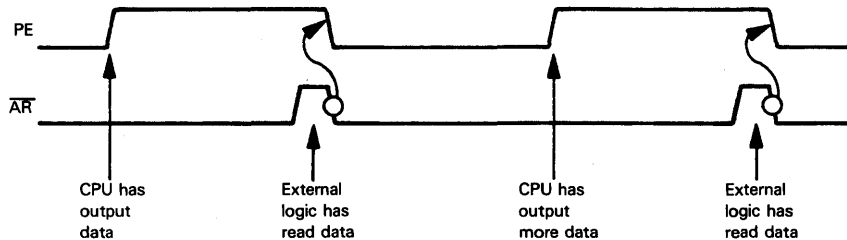
Figure 2-15. PD1680 Handshaking for Data Output

The most important point to note is that there is no control bit which specifies data input mode or data output mode. Thus, the signal sequences we described for data input and those we are about to describe for data output occur automatically; the input or output mode is purely a function of CPU and external logic interpretation.

Whenever the CPU outputs data to the PD1680, the arrival of data forces PE output high. If PD0 - PD15 has been configured as two 8-bit ports, then the arrival of a single data byte to either port will cause PE to be output high. If PD0 - PD15 is configured as a single 16-bit I/O port, then PD will not be output high until two bytes of data have been received from the CPU by the PD1680.

Once PE is output high, nothing more happens until external logic responds. External logic cannot tell by the simple inspection of any control signals whether a data input operation or a data output operation is in progress. It is up to you, when designing your system, to dedicate CP1680 devices to input or output; or you must generate your own identification logic in the event that a CP1680 IOB is bidirectional. In Figure 2-15 we simply assume that external logic knows data is to be read, and knows whether the data is 16 bits or 8 bits wide. Furthermore, if the data is 8 bits wide, external logic must know which 8 bits to read. In any event, when external logic has completed its undefined operations, it must input $\overline{AR}$ low. The high-to-low transition of $\overline{AR}$ forces PE low again, and if interrupts are enabled, an interrupt will be requested via $\overline{INTRQ}$. When the CPU acknowledges the interrupt by outputting INTAK via BC1, BC2, and BDIR, the PD1680 uses the INTAK pulse to reset $\overline{INTRQ}$ high.

The method used by external logic to reset $\overline{AR}$ high again is undefined. In Figure 2-15, we show PE going high as the trigger which external logic uses to reset $\overline{AR}$ high. This is clearly a viable scheme; PE will not go high again until fresh data has been output, at which point it is safe to assume that the CPU knows prior data has been read by external logic. It would be equally viable for external logic to hold $\overline{AR}$ continuously low, transmitting a short, high pulse whenever it reads data. This may be illustrated as follows:

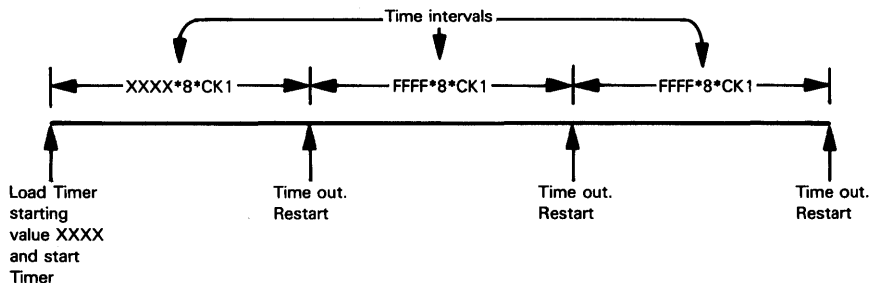


Because there are no control signals which identify the PD1680 operating in input mode or output mode, there is no straightforward scheme for handling bidirectional data transfers with a single PD1680 device.

THE CP1680 INTERVAL TIMER

The CP1680 has very elementary interval timer logic. A 16-bit Timer register, addressed as two separate 8-bit locations, decrements once every eight CK1 pulses, providing the timer has been enabled. You enable and disable timer logic via Control register bit 5. As a separate event, timer interrupts may be disabled via Control register bit 4. If timer interrupts are enabled, then when the timer decrements to 0, an interrupt request will occur. (Timer interrupt logic is described with other CP1680 interrupt logic later in this chapter.) If timer interrupts are not enabled, then the timer itself is effectively disabled, since you cannot test any timer status flag to see if the timer timed out; nor can you accurately read the contents of the Timer registers on the fly, since there is no protection against reading timer contents while it is in the process of being decremented.

The only timer programmable option you have is to load an initial value before the timer is enabled. The timer has no buffer; therefore, once it times out it begins decrementing again, if still enabled, beginning with the value FFFF₁₆. This may be illustrated as follows:



The only accurate long time intervals you can compute are exact multiples of $FFFF_{16} \cdot 8 \cdot CK1$.

The CP1600A uses a 4MHz two-phase clock, which generates a 500 nanosecond cycle time. Thus, $CK1$ equals 500 nanoseconds, and long CP1600A time intervals must be an exact multiple of 262.144 milliseconds — the time it will take for the counter to decrement from $FFFF_{16}$ to 0000.

The CP1600 uses a 3.3MHz two-phase clock, which generates a 600 nanosecond cycle time; therefore, long time intervals must be exact multiples of 314.572 milliseconds.

The CP1610, which runs on a 2MHz two-phase clock and has a one microsecond cycle time, will compute long time intervals that are exact multiples of 524.288 milliseconds.

You cannot attempt to generate clock periods that are multiples of shorter time intervals by loading some initial value into the timer following each time out; an unknown amount of time will elapse between the interval timer interrupt occurring and being acknowledged. The length of this unknown period of time will depend on the number of non-interruptible instructions which may be executing in sequence when the interrupt request first occurs, plus any higher priority interrupts which may exist. Therefore, if you load an initial value into the timer, it should be to compute an isolated time interval only. Here is an appropriate instruction sequence:

```
MVI    IOB,R0    ;INPUT CONTROL REGISTER CONTENTS
ANDI    CFH,R0    ;ZERO BITS 4 AND 5
MVO     R0,IOB    ;RETURN TO CONTROL REGISTER
MVII    2AH,R0    ;TRANSMIT LOW-ORDER TIMER
MVO     R0,IOB+3  ;INITIAL BYTE
MVII    34H,R0    ;TRANSMIT HIGH-ORDER TIMER
MVO     R0,IOB+4  ;INITIAL BYTE
MVI     IOB,R0    ;LOAD PRIOR CONTROL REGISTER CONTENTS
ADDI    30H,R0    ;SET BITS 4 AND 5
MVO     R0,IOB    ;START TIMER
```

The instruction sequence above begins with three instructions that load the CP1680 Control register contents into Register R0. Bits 4 and 5 are zeroed, then the result is returned to the Control register. Thus, the timer and timer interrupts are disabled. We do not bother with an SDBD instruction. Since the data source is eight bits wide, only the low-order byte of Register R0 will be significant. This being the case, we can use an 8-bit immediate AND mask to modify Register R0 contents before returning the low-order byte to the Control register.

Next, we load the initial timer value, one byte at a time, into Register R0. Each byte is written out to the appropriate half of the Control register. Once again we do not need to use the SDBD instruction. Since an 8-bit data path connects the CPU to the 1680 IOB, only the low-order byte of Register R0 will be significant during the data output.

Finally, we start the timer by loading Control register contents into Register R0, setting bits 4 and 5 to 1 and writing back the result.

When you write into the Timer registers, you clear any timer interrupt requests which may at that time be pending.

CP1680 INTERRUPT LOGIC

A CP1680 IOB will generate an interrupt request by outputting a low signal at $\overline{INTRQ}$ if any one of these three conditions occurs:

- 1) A low input at $\overline{ERROR}$. External logic can request an interrupt via the CP1680 using the $\overline{ERROR}$ input.
- 2) The $\overline{AR}$ handshaking control input makes a high-to-low transition. This is illustrated in Figures 2-14 and 2-15.
- 3) The Interval Timer decrements from 1 to 0.

Recall that there are two separate interrupt enable/disable control bits in the Control register. One control bit applies to the Interval Timer, while the other control bit applies to both the $\overline{AR}$ handshaking and $\overline{ERROR}$ interrupts.

Interrupt priorities among the three sources within a single CP1680 IOB are as follows:

$\overline{ERROR}$	highest
$\overline{AR}$	handshaking
Timer	lowest

When more than one CP1680 IOB is present in a CP1600 microcomputer system, then daisy chain priority is implemented using the MSKI input signal and the MSKO output signal. Signal connections are illustrated in Figure 2-13. **The manner in which interrupt priorities are handled by the CP1680 is a little unusual.**

Two or more CP1680 devices may combine their interrupt request signals, which are wired ORed and input to the CP1600 via $\overline{INTRQ}$. The CP1600 acknowledges an interrupt via the INTAK combination of BC1, BC2, and BDIR. We de-

scribed this process earlier in the chapter. All CP1680 devices simultaneously receive the INTAK combination; however, a CP1680 which is acknowledged raises its IMSKO signal high, causing it to become the IMSKI input to the next CP1680 in the daisy chain. Any device that receives a high IMSKI input ignores the interrupt acknowledge. Thus, only the highest priority, interrupt requesting CP1680 device in the daisy chain will process the interrupt acknowledge. However, it takes a finite amount of time for IMSKO high signals to propagate as IMSKI signals, and thus ripple through the daisy chain. Consequently, a maximum of eight CP1680 devices may be present in the daisy chain. A ninth device will receive its IMSKI high signal too late and will respond to an interrupt acknowledge.

CP1680 IOB devices maintain their interrupt priority status until they receive a high TCI pulse. At that time, prior interrupt priorities are reset at all devices, and new priority arbitration begins. Thus, **when using CP1680 IOB devices, you are required to end all interrupt service routines by executing a TCI instruction.**

Note that if one CP1680 IOB has more than one active interrupt request (for example, an ERROR interrupt request and a timer interrupt request), then this internal interrupt priority will take precedence over the daisy chain interrupt priority. That is to say, the ERROR interrupt request will be acknowledged and serviced first. After the next TCI instruction is executed, the timer interrupt request will be serviced before any interrupt request from a lower priority CP1680 device is acknowledged.

Every CP1680 device has three 8-bit Interrupt Vector registers, one dedicated to each of the three interrupt sources. These three Interrupt Vector registers were illustrated earlier in the chapter. **Following an interrupt acknowledge, when the IAB combination appears at BC1, BC2, and BDIR, the contents of the Interrupt Vector register for the highest priority active interrupt will be returned to the CPU.** Interrupt acknowledge timing is illustrated in Figure 2-9. At the interrupt service location a Jump-to-Subroutine instruction will probably be stored. Since the Jump-to-Subroutine object code is three words long, a maximum of 85 interrupts can be originated in the first 256 words of memory. This is more than sufficient, since only eight CP1680 devices with 24 interrupts can be supported in a single daisy chain.

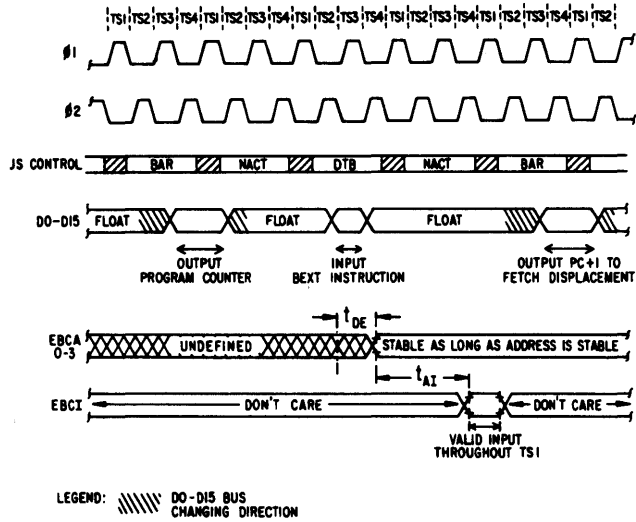
DATA SHEETS

This section contains specific electrical and timing data for the following devices:

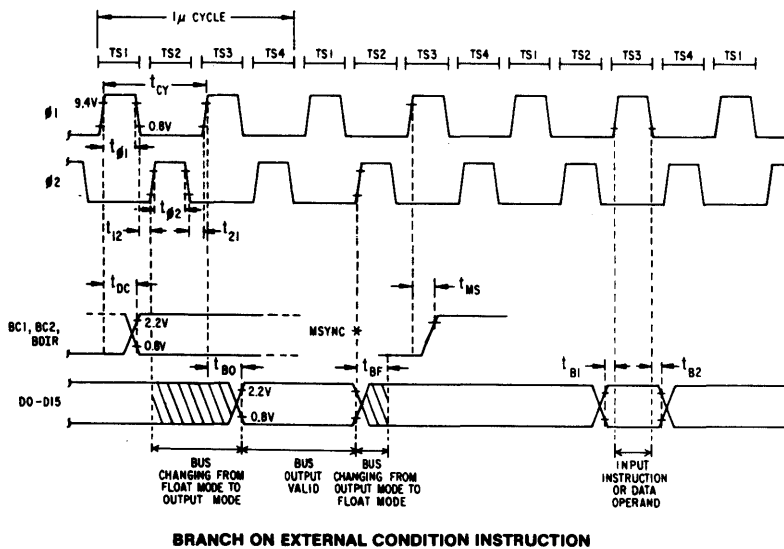
- CP1600 CPU
- CP1600A CPU
- CP1610 CPU
- IOB1680 I/O Buffer

CP1600·CP1600A·CP1610

BUS TIMING DIAGRAM



TYPICAL INSTRUCTION SEQUENCE



CP1600

ELECTRICAL CHARACTERISTICS (CP1600)

Maximum Ratings*

V_{DD} , V_{CC} , GND and all other input/output voltages
with respect to V_{BB} -0.3V to +18.0V
Storage Temperature -55°C to +150°C
Operating Temperature 0°C to +70°C

*Exceeding these ratings could cause permanent damage to these devices. Functional operation at these conditions is not implied—operating conditions are specified below.

Standard Conditions: (unless otherwise noted)

$V_{DD}=+12V\pm5\%$, 70mA(typ) , 110mA(max.) $V_{BB}=-3V\pm10\%$, 0.2mA(typ) , 2mA(max.)
 $V_{CC}=+5V\pm5\%$, 12mA(typ) , 25mA(max.) Operating Temperature (T_A)=0°C to +70°C

Characteristic	Sym	Min	Typ**	Max	Units	Conditions	
DC CHARACTERISTICS							
Clock Inputs							
High	V _{IHC}	10.4	—	V _{DD}	V		
Low	V _{ILC}	0	—	0.6	V		
Logic Inputs							
Low	V _{IL}	0	—	0.65	V		
High (All Lines except BDRDY)	V _{IH}	2.4	—	V _{CC}	V		
High (Bus Data Ready Line See Note)	V _{IHB}	3.0	—	V _{CC}	V		
Logic Outputs							
High	V _{OH}	2.4	V _{CC}	—	V	I _{OH} = 100μA	
Low (Data Bus Lines DO-D15)	V _{OL}	—	—	0.5	V	I _{OL} = 1.6mA	
Low (Bus Control Lines, BC1,BC2,BDIR)	V _{OL}	—	—	0.45	V	I _{OL} = 2.0mA	
Low (All Others)	V _{OL}	—	—	0.45	V	I _{OL} = 1.6mA	
AC CHARACTERISTICS							
Clock Pulse Inputs, φ1 or φ2							
Pulse Width	t _{φ2} , t _{φ2}	120	—	—	ns		
Skew (φ1, φ2 delay)	t ₁₂ , t ₂₁	0	—	—	ns		
Clock Period	t _{cy}	0.3	—	2.0	μs		
Rise & Fall Times	t _r , t _f	—	—	15	ns		
Master SYNC:							
Delay from φ	t _{ms}	—	—	30	ns		
DO-D15 Bus Signals							
Output delay from φ1 (float to output)	t _{BO}	—	—	120	ns	1 TTL Load & 25 pF 	
Output delay from φ2 (output to float)	t _{BF}	—	50	—	ns		
Input setup time before φ1	t _{B1}	0	—	—	ns		
Input hold time after φ1	t _{B2}	10	—	—	ns		
Bus Control Signals							
BC1,BC2,BDIR							
Output delay from φ1	t _{DC}	—	—	120	ns		
BUSAK Output delay from φ1	t _{BU}	—	150	—	ns		
TCI Output delay from φ1	t _{TO}	—	200	—	ns		
TCI Pulse Width	t _{TW}	—	300	—	ns		
EBCA output delay from BEXT input	t _{DE}	—	—	150	ns		
EBCA wait time for EBCI input	t _{AI}	—	—	400	ns		
CAPACITANCE							
φ1, φ2 Clock Input capacitance	C _{φ1} , C _{φ2}	—	20	30	pF	T _A = +25°C; V _{DD} = +12V; V _{CC} = +5V V _{BB} = -3V; t _{φ1} t _{φ2} = 120ns	
Input Capacitance							
DO-D15	C _{IN}	—	6	12	pF		
All Other	—	—	5	10	pF		
Output Capacitance							
DO-D15 in high impedance state	C _O	—	8	15	pF		

**Typical values are at +25°C and nominal voltages.

NOTE:

The Bus Data Ready (BDRDY) line is sampled during time period TS1 after a BAR or ADAR bus control signal. BDRDY must go low requesting a wait state 50 ns before the end of TS1 and remain low for 50 ns minimum. BDRDY may go high asynchronously. In response to BDRDY, the CPU will extend bus cycles by adding additional microcycles up to a maximum of 40 μs duration.

CP1600A

ELECTRICAL CHARACTERISTICS (CP1600A)

Maximum Ratings*

V_{DD} , V_{CC} , GND and all other input/output voltages
with respect to V_{BB} -0.3V to +18.0V
Storage Temperature -55°C to +150°C
Operating Temperature 0°C to +70°C

*Exceeding these ratings could cause permanent damage to these devices. Functional operation at these conditions is not implied—operating conditions are specified below.

Standard Conditions: (unless otherwise noted)

$V_{DD}=+12V\pm5\%$, 70mA(typ) , 140mA(max.) $V_{BB}=-3V\pm10\%$, 0.2mA(typ) , 2mA(max.)
 $V_{CC}=+5V\pm5\%$, 12mA(typ) , 25mA(max.) Operating Temperature (T_A)=0°C to +70°C

Characteristic	Sym	Min	Typ**	Max	Units	Conditions	
DC CHARACTERISTICS							
Clock Inputs							
High	V _{IHC}	10.4	—	V _{DD}	V		
Low	V _{ILC}	0	—	0.6	V		
Logic Inputs							
Low	V _{IL}	0	—	0.65	V		
High (All Lines except BDRDY)	V _{IH}	2.4	—	V _{CC}	V		
High (Bus Data Ready Line See Note)	V _{IHB}	3.0	—	V _{CC}	V		
Logic Outputs							
High	V _{OH}	2.4	V _{CC}	—	V	I _{OH} = 100μA	
Low (Data Bus Lines DO-D15)	V _{OL}	—	—	0.5	V	I _{OL} = 1.6mA	
Low (Bus Control Lines, BC1,BC2,BDIR)	V _{OL}	—	—	0.45	V	I _{OL} = 2.0mA	
Low (All Others)	V _{OL}	—	—	0.45	V	I _{OL} = 1.6mA	
AC CHARACTERISTICS							
Clock Pulse Inputs, φ1 or φ2							
Pulse Width	t _{φ2} , t _{φ2}	95	—	—	ns		
Skew (φ1, φ2 delay)	t ₁₂ , t ₂₁	0	—	—	ns		
Clock Period	t _{cy}	0.25	—	2.0	μs		
Rise & Fall Times	t _r , t _f	—	—	15	ns		
Master SYNC:							
Delay from φ	t _{ms}	—	—	30	ns		
DO-D15 Bus Signals							
Output delay from φ1 (float to output)	t _{BO}	—	—	95	ns	1 TTL Load & 25 pF 	
Output delay from φ2 (output to float)	t _{BF}	—	50	—	ns		
Input setup time before φ1	t _{B1}	0	—	—	ns		
Input hold time after φ1	t _{B2}	10	—	—	ns		
Bus Control Signals							
BC1,BC2,BDIR							
Output delay from φ1	t _{DC}	—	—	200	ns		
BUSAK Output delay from φ1	t _{BU}	—	150	—	ns		
TCI Output delay from φ1	t _{TO}	—	200	—	ns		
TCI Pulse Width	t _{TW}	—	300	—	ns		
EBCA output delay from BEXT input	t _{DE}	—	—	150	ns		
EBCA wait time for EBCI input	t _{AI}	—	—	400	ns		
CAPACITANCE							
φ1, φ2 Clock Input capacitance	C _{φ1} , C _{φ2}	—	20	30	pF	T _A = +25°C; V _{DD} = +12V; V _{CC} = +5V V _{BB} = -3V; t _{φ1} t _{φ2} = 120ns	
Input Capacitance							
DO-D15	C _{IN}	—	6	12	pF		
All Other	—	—	5	10	pF		
Output Capacitance							
DO-D15 in high impedance state	C _O	—	8	15	pF		

**Typical values are at +25°C and nominal voltages.

NOTE:

The Bus Data Ready (BDRDY) line is sampled during time period TS_1 after a BAR or ADAR bus control signal. BDRDY must go low requesting a wait state 50 ns before the end of TS_1 and remain low for 50 ns minimum. BDRDY may go high asynchronously. In response to BDRDY, the CPU will extend bus cycles by adding additional microcycles up to a maximum of 40 μsec duration.

CP1610

ELECTRICAL CHARACTERISTICS (CP1610)

Maximum Ratings*

V_{DD} , V_{CC} , GND and all other input/output voltages
with respect to V_{BB} -0.3V to +18.0V
Storage Temperature -55°C to +150°C
Operating Temperature 0°C to +70°C

*Exceeding these ratings could cause permanent damage to these devices. Functional operation at these conditions is not implied—operating conditions are specified below.

Standard Conditions: (unless otherwise noted)

V_{DD} =+11V±5%, 70mA(typ), 110mA(max.) V_{BB} =-3V±10%, 0.2mA(typ), 2mA(max.)
 V_{CC} =+5V±5%, 12mA(typ), 25mA(max.) Operating Temperature (T_A)=0°C to +70°C

Characteristic	Sym	Min	Typ**	Max	Units	Conditions	
DC CHARACTERISTICS							
Clock Inputs							
High	V _{IHC}	10.0	—	V _{DD}	V	V _{IHC} = V _{DD} -1	
Low	V _{ILC}	0	—	0.6	V		
Input current	—	—	—	15	mA		
Logic Inputs							
Low	V _{IL}	0	—	0.65	V		
High (All Lines except BDRDY)	V _{IH}	2.4	—	V _{CC}	V		
High (Bus Data Ready Line See Note)	V _{IHB}	3.0	—	V _{CC}	V		
Logic Outputs							
High	V _{OH}	2.4	V _{CC}	—	V	I _{OH} = 100μA	
Low (Data Bus Lines DO-D15)	V _{OL}	—	—	0.5	V	I _{OL} = 1.6mA	
Low (Bus Control Lines, BC1,BC2,BDIR)	V _{OL}	—	—	0.45	V	I _{OL} = 2.0mA	
Low (All Others)	V _{OL}	—	—	0.45	V	I _{OL} = 1.6mA	
AC CHARACTERISTICS							
Clock Pulse Inputs, φ1 or φ2							
Pulse Width	t _{φ2} , t _{φ2}	250	—	—	ns	1 TTL Load & 25 pF 	
Skew (φ1, φ2 delay)	t ₁₂ , t ₂₁	0	—	—	ns		
Clock Period	t _{cy}	0.5	—	2.0	μs		
Rise & Fall Times	t _r , t _f	—	—	15	ns		
Master SYNC:							
Delay from φ	t _{ms}	—	—	30	ns		
DO-D15 Bus Signals							
Output delay from φ1 (float to output)	t _{BO}	—	—	200	ns		
Output delay from φ2 (output to float)	t _{BF}	—	50	—	ns		
Input setup time before φ1	t _{B1}	0	—	—	ns		
Input hold time after φ1	t _{B2}	10	—	—	ns		
Bus Control Signals							
BC1,BC2,BDIR							
Output delay from φ1	t _{DC}	—	—	200	ns		
BUSAK Output delay from φ1	t _{BU}	—	150	—	ns		
TCI Output delay from φ1	t _{TO}	—	200	—	ns		
TCI Pulse Width	t _{TW}	—	300	—	ns		
EBCA output delay from BEXT input	t _{DE}	—	—	150	ns		
EBCA wait time for EBCI input	t _{AI}	—	—	400	ns		
CAPACITANCE							
φ1, φ2 Clock Input capacitance	C _{φ1} , C _{φ2}	—	20	30	pF	T _A = +25°C; V _{DD} = +12V; V _{CC} = +5V; V _{BB} = -3V; t _{φ1} t _{φ2} = 120ns	
Input Capacitance							
DO-D15	C _{IN}	—	6	12	pF		
All Other	—	—	5	10	pF		
Output Capacitance							
DO-D15 in high impedance state	C _D	—	8	15	pF		

**Typical values are at +25°C and nominal voltages.

NOTE:

The Bus Data Ready (BDRDY) line is sampled during time period TS1 after a BAR or ADAR bus control signal. BDRDY must go low requesting a wait state 50 ns before the end of TS1 and remain low for 50 ns minimum. BDRDY may go high asynchronously. In response to BDRDY, the CPU will extend bus cycles by adding additional microcycles up to a maximum of 40 μsec duration.

IOB1680

ELECTRICAL CHARACTERISTICS

Maximum Ratings*

V_{DD} and V_{CC} and all other input/output voltages with respect to GND	-0.3V to +18V
Storage Temperature	-55°C to +150°C
Operating Temperature	0°C to +70°C

*Exceeding these ratings could cause permanent damage. Functional operation of this device at these conditions is not implied—operating ranges are specified below.

Standard Conditions (unless otherwise noted)

All voltages referenced to GND

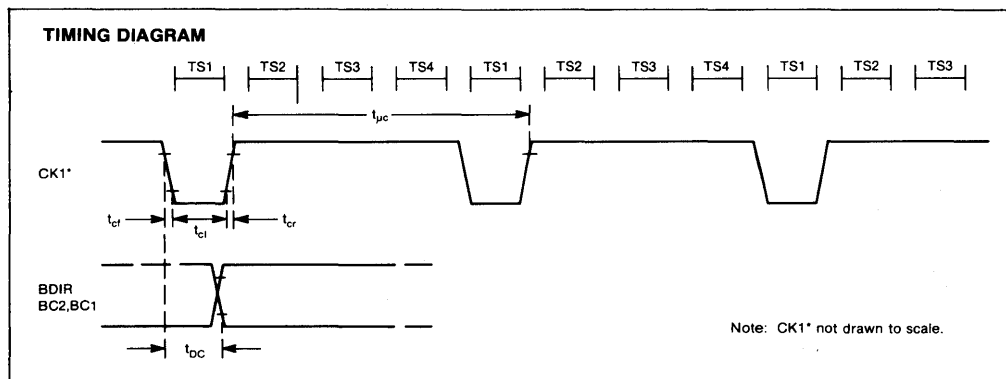
$V_{DD} = +12V \pm 5\%$

$V_{CC} = +5V \pm 5\%$

Operating Temperature (T_A) = 0°C to +70°C

Characteristic	Symbol	Min	Typ**	Max	Unit	Condition
DC CHARACTERISTICS						
Clock Input: High	V_{ihc}	2.4	—	V_{DD}	V	
Low	V_{ilc}	0	—	.5	V	
Logic Inputs: High	V_{ih}	2.4	—	V_{CC}	V	
Low	V_{il}	0	—	.65	V	
Logic Outputs: High	V_{oh}	2.4	V_{CC}	—	V	$I_{oh} = 100\mu A$
Low	V_{ol}	—	—	.5	V	$I_{ol} = 1.6mA$
AC CHARACTERISTICS						
Clock Inputs						
$\overline{CK1}$ Clock period	$t_{\mu c}$	0.4	—	4.0	μs	
Clock width	t_{cl}	70	—	—	ns	
Rise & Fall times	t_{cr}, t_{cf}	—	—	10	ns	
CAPACITANCE ($T_A = 25^\circ C$, $V_{DD} = +12V$, $V_{CC} = +5V$)						
Input Capacitance: D0-D7	C_{in}	—	6	12	pF	$V_{in} = 0V$
All others		—	5	10	pF	$V_{in} = 0V$
Output Capacitance:	C_{out}	—	8	15	pF	

**Typical values are at +25°C and nominal voltages.



CIRCUIT DESCRIPTION

This circuit is designed to provide all the data buffering and control functions required when interfacing the Series 1600 Microprocessor System to a simple peripheral device. Data is transferred to and from the peripheral on 16 bidirectional lines, each of which can be considered to be an input or output. The transfer of information with the CP1600 is accomplished via an 8-bit highway, the 16-bits being transferred as two 8-bit bytes. The register addresses are assigned CP1600 memory locations, as follows (N is an arbitrary starting address):

Register Address Description

N	Control Register
N + 1	Data Register Low Order 8-bits
N + 2	Data Register High Order 8-bits
N + 3	Timer Low Order 8-bits
N + 4	Timer High Order 8-bits
N + 5	Peripheral Interrupt Address Vector
N + 6	Timer Interrupt Address Vector
N + 7	Error Interrupt Address Vector